

Vorlesung (WS 2016/17) *Softwarekonstruktion*

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

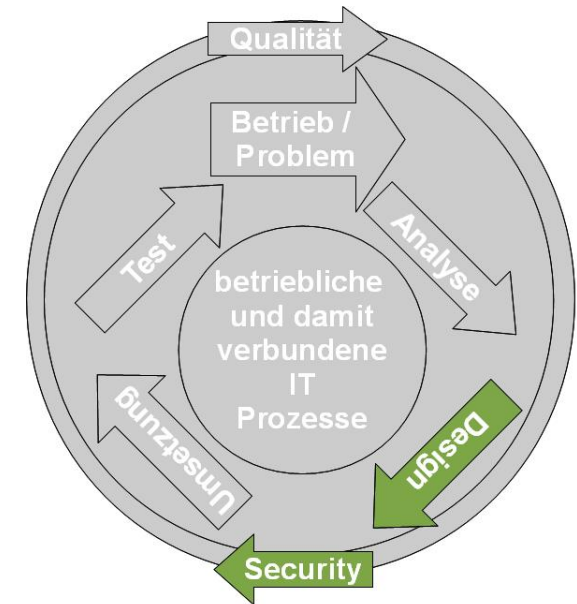
1.4: Eclipse Modeling Framework (EMF) und Meta-Modellierung

v. 14.11.2016

Einordnung

1.5 Eclipse Modeling Framework (EMF)

- Modellgetriebene SW-Entwicklung
 - Einführung
 - Modellbasierte Softwareentwicklung
 - OCL
 - Petrinetze
 - Metamodellierung und EMF
- Qualitätsmanagement
- Testen



Inkl Beiträge von Markus Bauer, Florian Lautenbacher, Stephan Roser.

Literatur:

- V. Gruhn: **MDA - Effektives Software-Engineering**. (s. Vorlesungswebseite)
- Kapitel 8.2

- Grundlegendes Verständnis über Konzepte von MDA und MDD mit EMF
- Erfahrung mit dem Umgang von Eclipse und EMF
- Verständnis über Metamodellierung und deren Anwendung in praktischen Softwareprojekten
- Verständnis über Metaprogrammierung

- **Vorheriger Abschnitt:** Grundlage und Techniken für modellbasierte Softwareentwicklung.
- **Dieser Abschnitt:**
 - Grundlagen der Metamodellierung und
 - technische Grundlage dafür: Eclipse Modeling Framework
 - Vorstellung der Standards → EMF
 - Kurzer Überblick über Metaprogrammierung

A model has three features [Stachowiak]:

1. mapping feature (“Abbildungsmerkmal”):

A model is always a model *of something*, that is, a mapping or representation of a natural or artificial original, which may in turn be a model itself.

2. reduction feature (“Verkürzungsmerkmal”):

A model usually does not capture *all* attributes of the original it represents, but only those which are relevant to its creator / user.

3. pragmatic feature (“Pragmatisches Merkmal”):

A model needs to be usable in place of an original with respect to some purpose.

```
de.jabc.projects.bookservice::web::search::BookSearchController
showScreen(keywords : String,books : String[]) : void
```

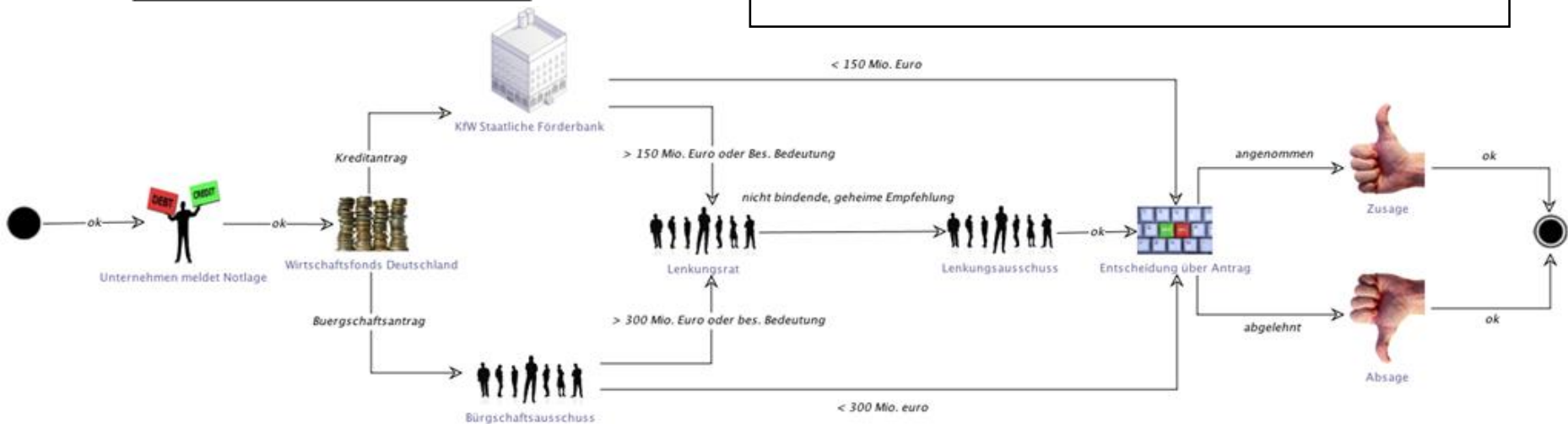
```
<<Service>>
de.jabc.projects.bookservice::services::BookService
searchBooks(keywords : String[]) : String[]
```

```
<<Entity>>
de.jabc.projects.bookservice::entities::Book
author : String
title : String
year : Integer
format() : String
```

```
entity Customer {
  String firstName
  String lastName
  String email
  Address shippingAddress
  Order* orders
}
```

```
entity Address {
  String line1
  String line2
  String city
  String postcode
  String state
  String country
}
```

```
entity Product {
  String description
  Double price
}
```

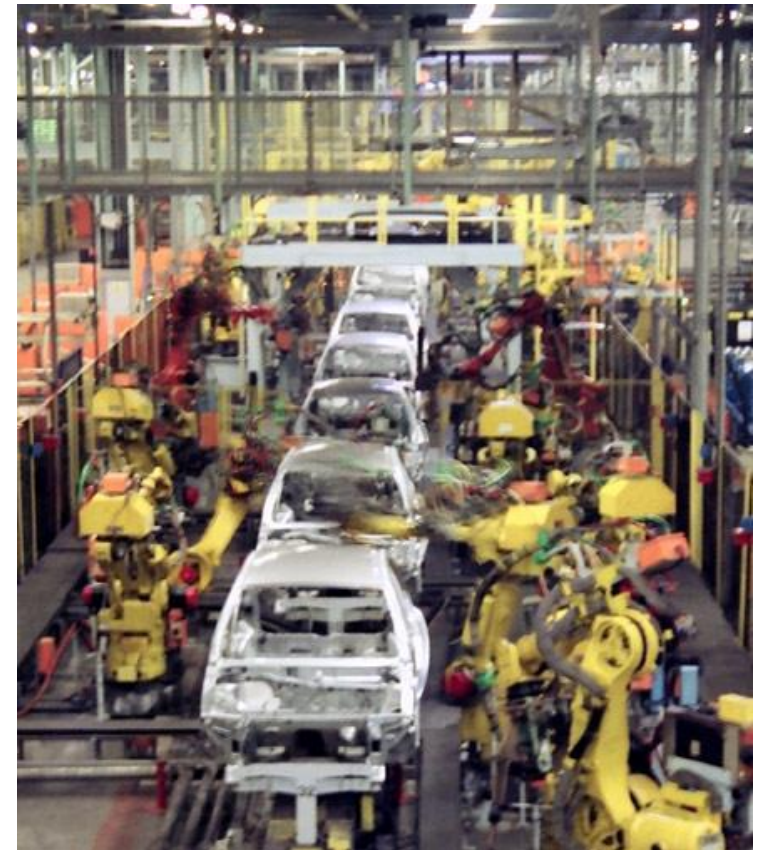


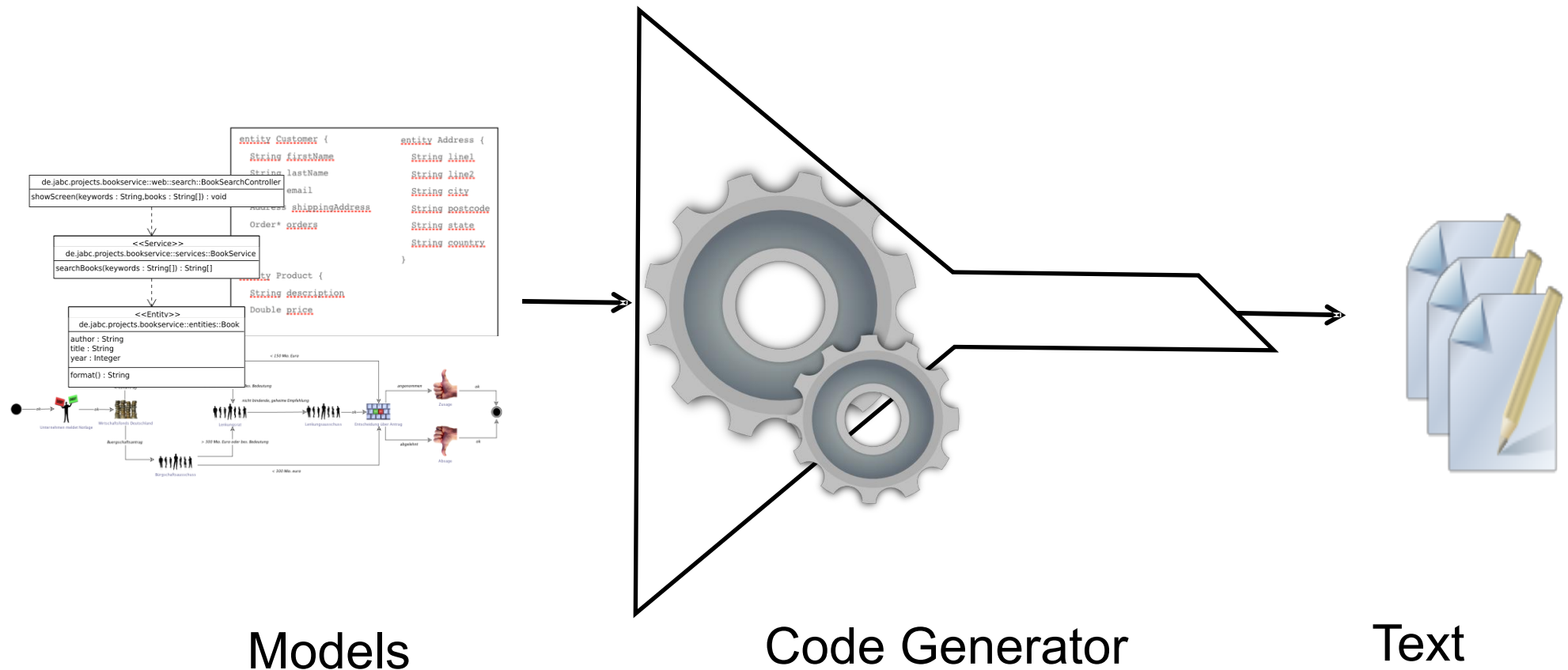
- models as **development artifacts** / „first citizens“
- **automatic code generation:**
 - to concrete target platform (Java EE, .NET, mobile phone, ...)
 - maybe preceded by: model refinement, model transformations, ...
- **domain specific languages:**
 - models use elements of a language tailored for a specific application domain
 - domain expertise can better be incorporated
 - code generators can be more restricted

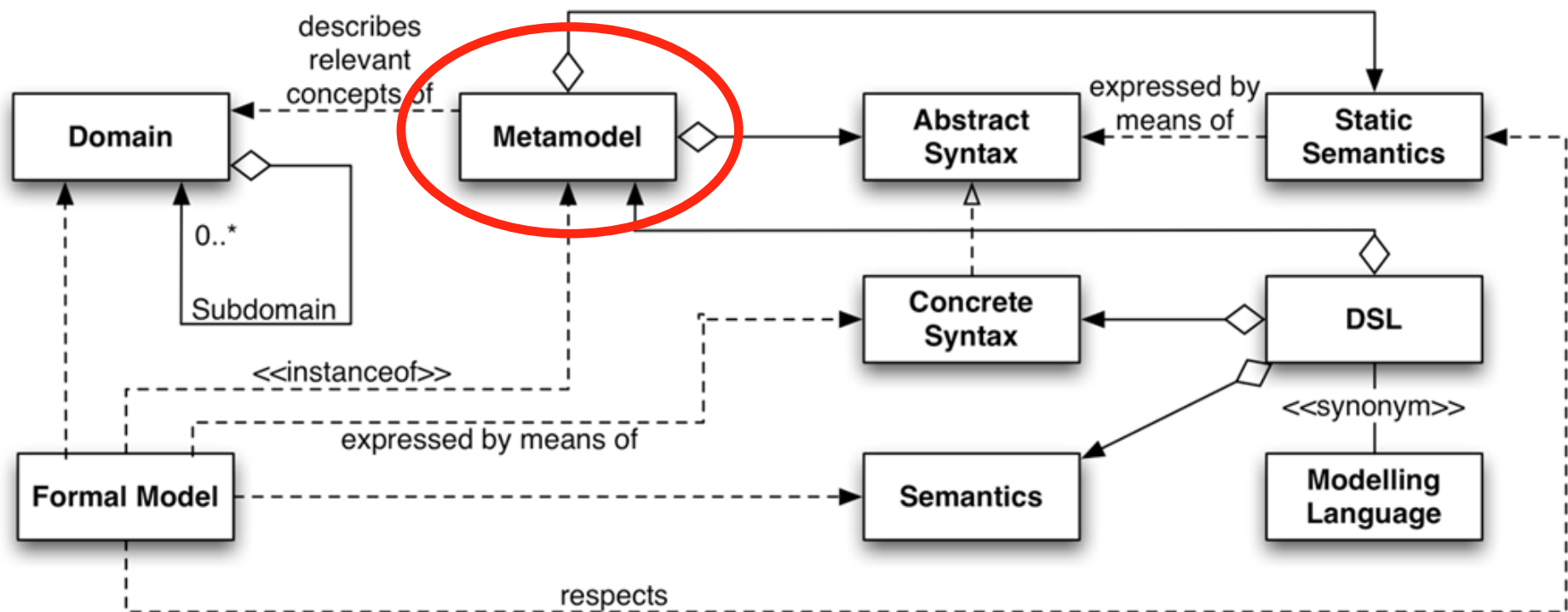


- higher code quality: no manual coding errors
- less repetitive work (e.g. boilerplate code)
- translate a model to another target platform = choice/implementation of an appropriate code generator

The Vision:
Complete, running
system „at the push of a
button“.





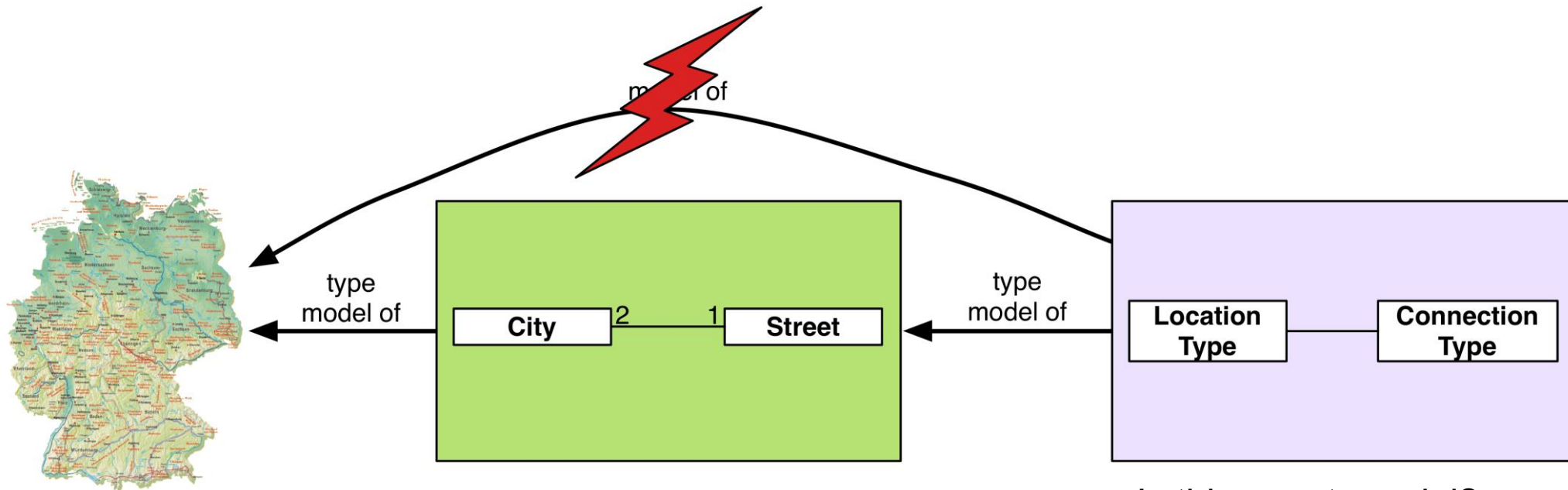


Source: [MDSD], S. 28

- Wikipedia:
Meta- (from Greek: μετά = "after", "beyond", "with", "adjacent", "self")
- [Kühne1]: ““meta” is used whenever an operation is applied twice”, e.g.
 - meta-discussion: a discussion about how to conduct a discussion
 - meta-learning: learning general learning strategies while learning a particular subject
 - meta-mathematics: applying mathematical methods to mathematics

[Kühne1]:

1. “Hence, any relation between two entities, which is going to be used to build up a meta-entity **must not be transitive.**“
2. “[...] in order to create metamodel we need **the same non-transitive relationship [...] twice.**“



Is this a metamodel?

Yes, as we apply the same non-transitive relation “type model of” twice!
→ anti-transitivity through double classification

Metamodeling Example 1: UML 1.x

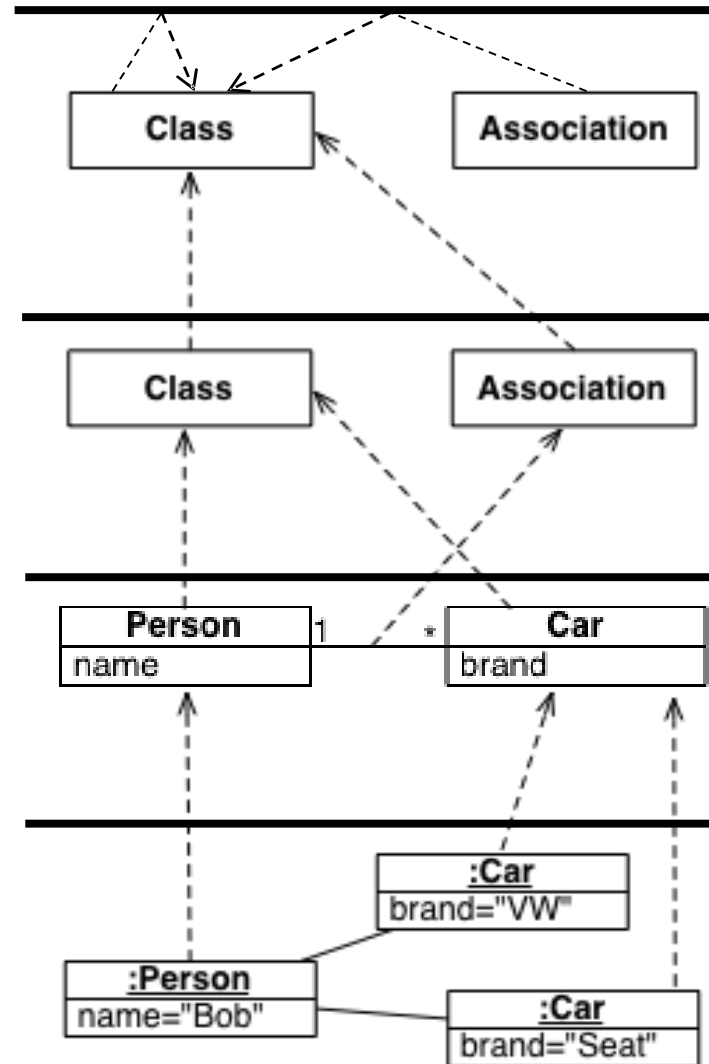
Softwarekonstruktion
WS 2016/17

Meta-Object
Facility (MOF)

UML 1.x

Concrete
Diagram

Concrete
Objects



„reflection“

M3: Metametamodel

More levels?
M2: Metamodel

M1: Model

M0: Model Instance

(E)BNF

```
rule: N ::= X1 .. Xn
```

M3: Metametamodel

**Java
grammar**

```
<ass_stmt> ::= ....
```

M2: Metamodel

Java Program

```
x = x+y;
```

M1: Model

JVM Class File

```
ILOAD 1  
ILOAD 3  
IADD
```

M0: Model Instance

Meta-Modellierung = Erstellung einer Modellierungssprache wird zum Gegenstandsbereich

Bestandteile:

- Syntax C (konkret) und Syntax A (abstrakt)
- Semantik S
- Syntaktische Abbildung $M_S : A \rightarrow C$
- Semantische Abbildung $M_C : A \rightarrow S$

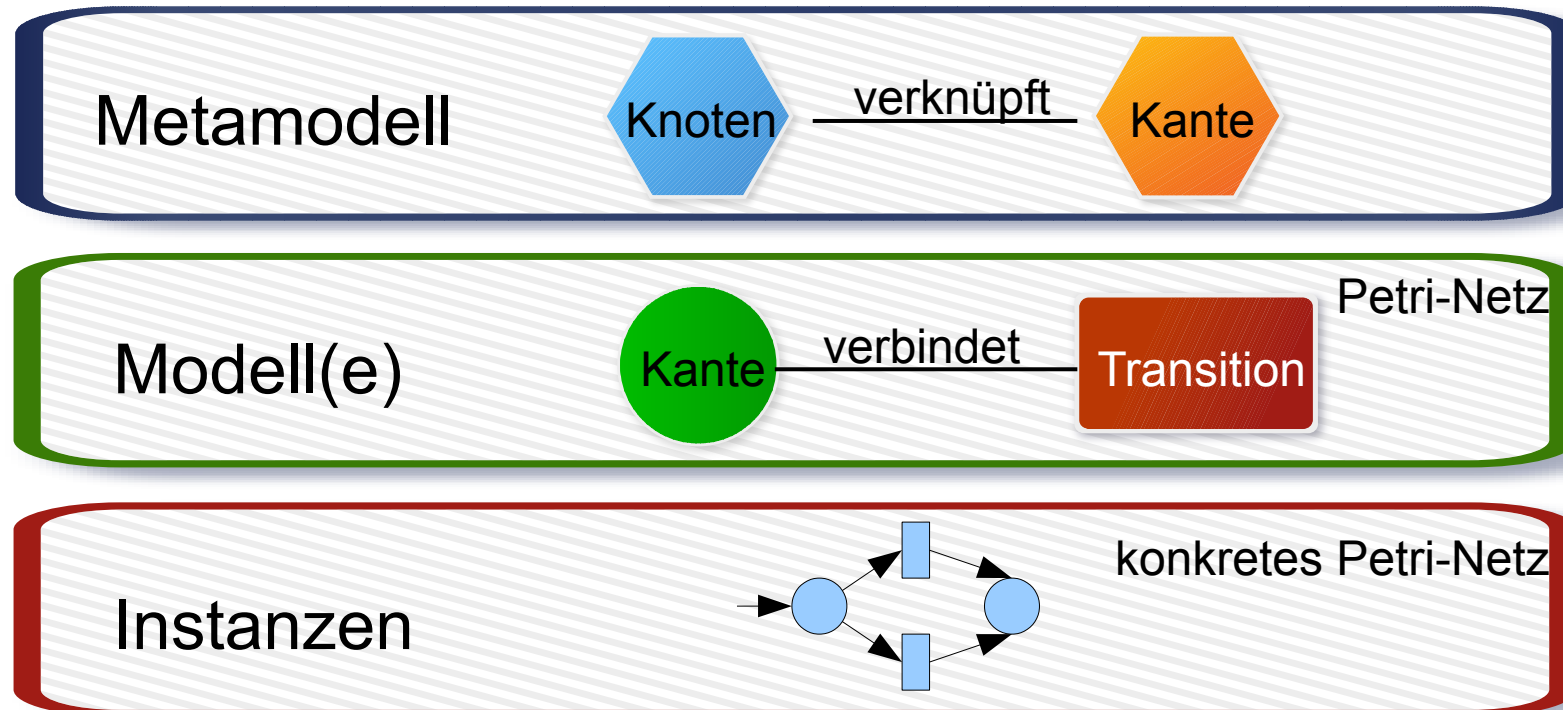
Semantik legt Bedeutung von Symbolen und deren Kombinationen fest. Bedeutung definiert in begrenztem Bereich (semantische Domäne)

Konkrete Syntax definiert Symbole für Modellierungssprache

- Textuelle Sprache
- Graphische Sprache
- Hybride (textuell und graphisch)

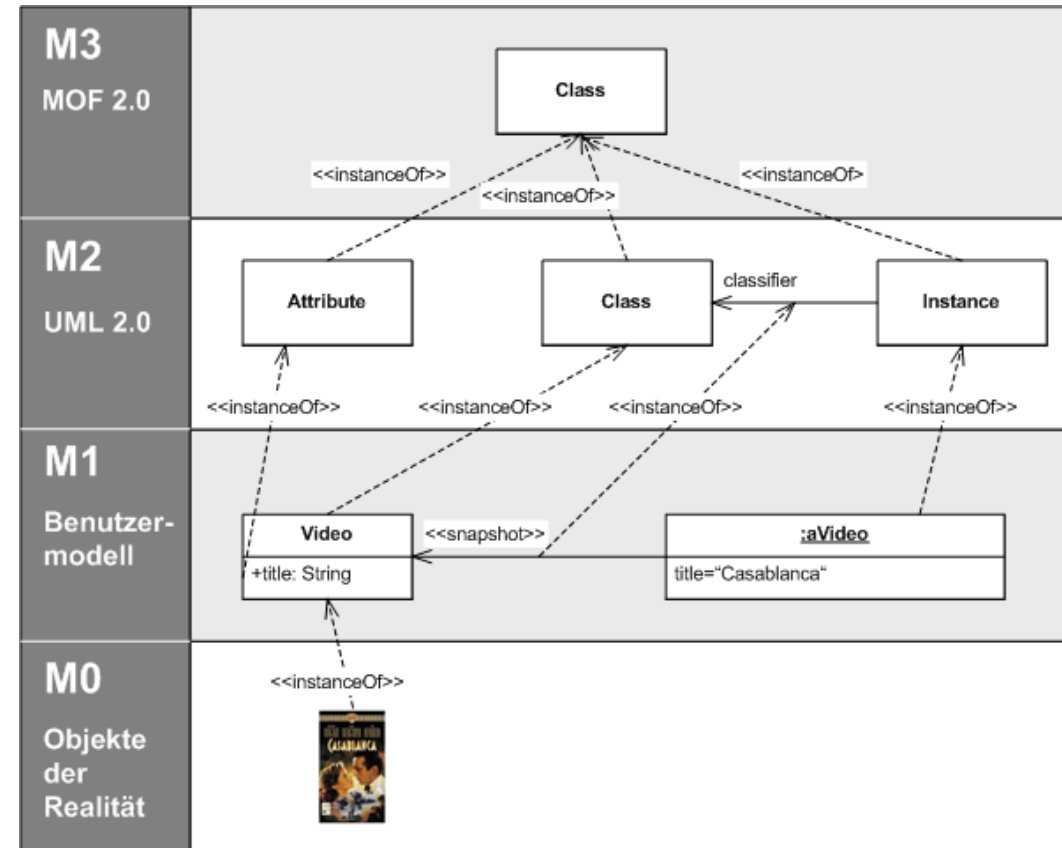
- Definiert Struktur der Symbole durch Konzepte, Beziehungen zwischen Konzepten und Integrationsbedingungen
- Erfasst **nur** Modellierungskonzepte (Abstraktion)
- Definition der abstrakten Syntax als:
 - Graphische Sprache: Metamodelle oder Graphgrammatiken
 - Textuelle Sprache: (Baum-)Grammatiken

Metamodell = mehrstufige, geschichtete Klassifizierung

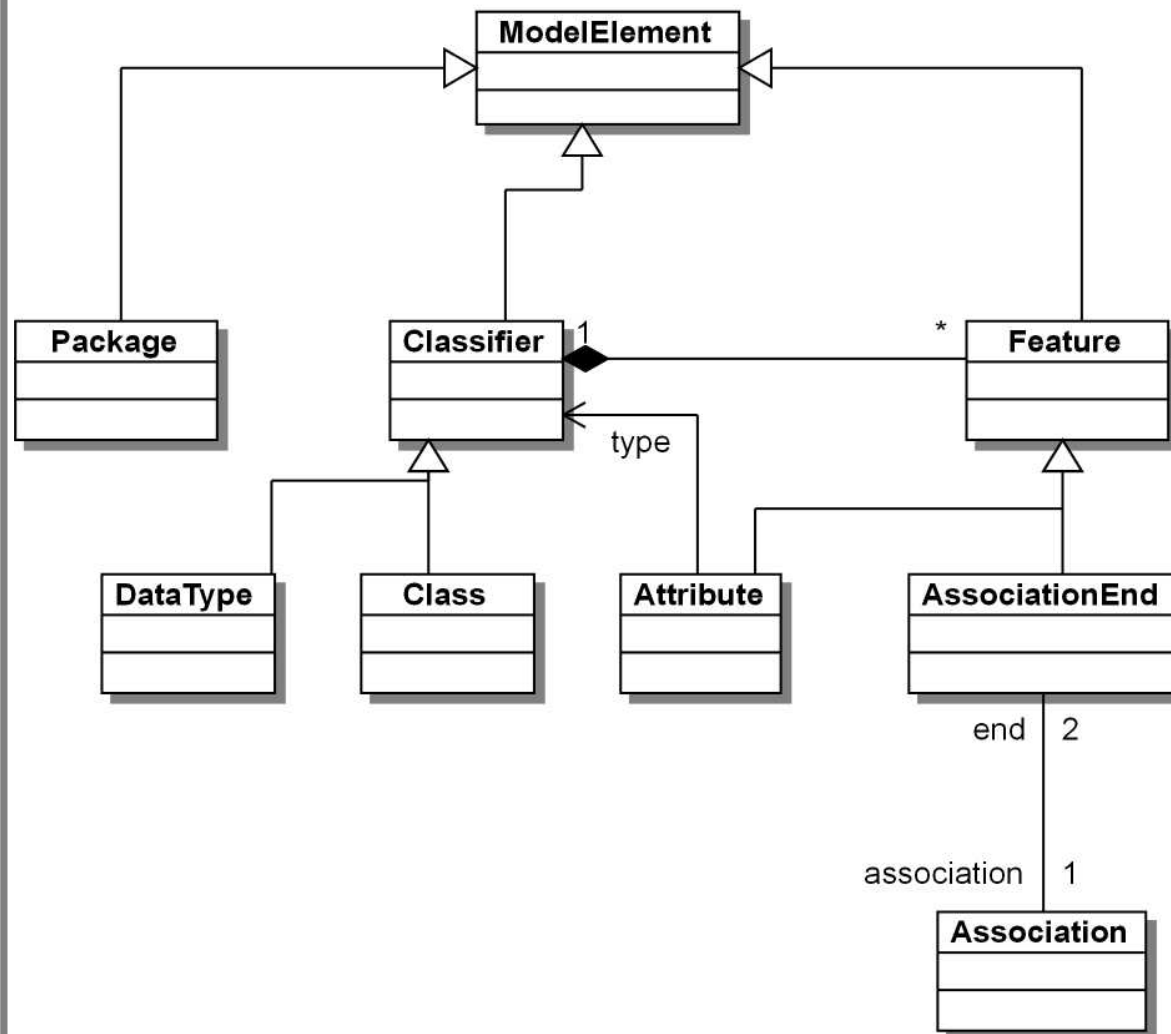


Object Management Group (OMG) Standard

- M0-Ebene - Konkret
- M1-Ebene - Modelle
- M2-Ebene – Meta-Modelle
 - Definieren, wie die Modelle aufgebaut und strukturiert sind.
- M3-Ebene - Meta-Meta-Modelle (bzw. MOF-Ebene)



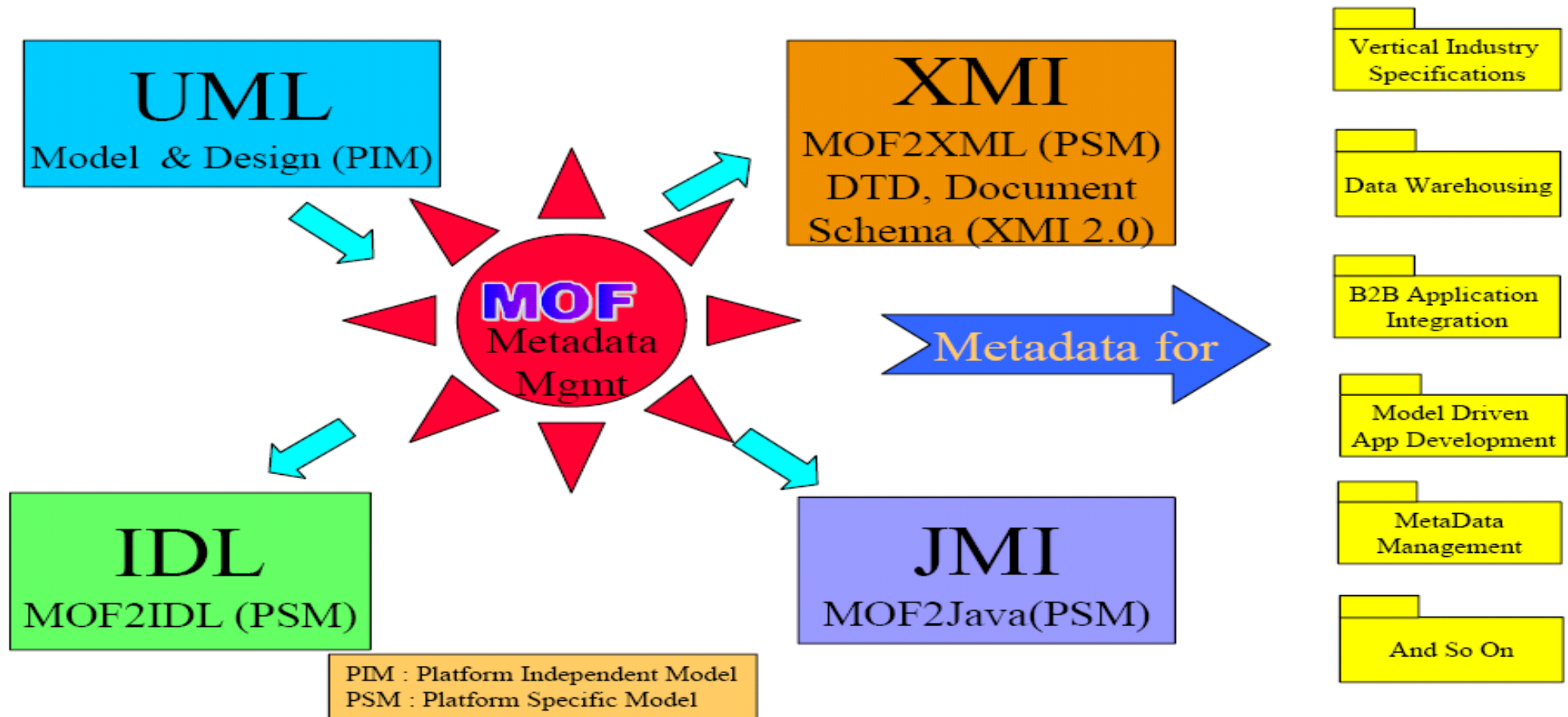
MOF Metamodel (vereinfacht)



Selbstreferenzierendes
Metamodell = Metamodell
beschreibt sich selbst

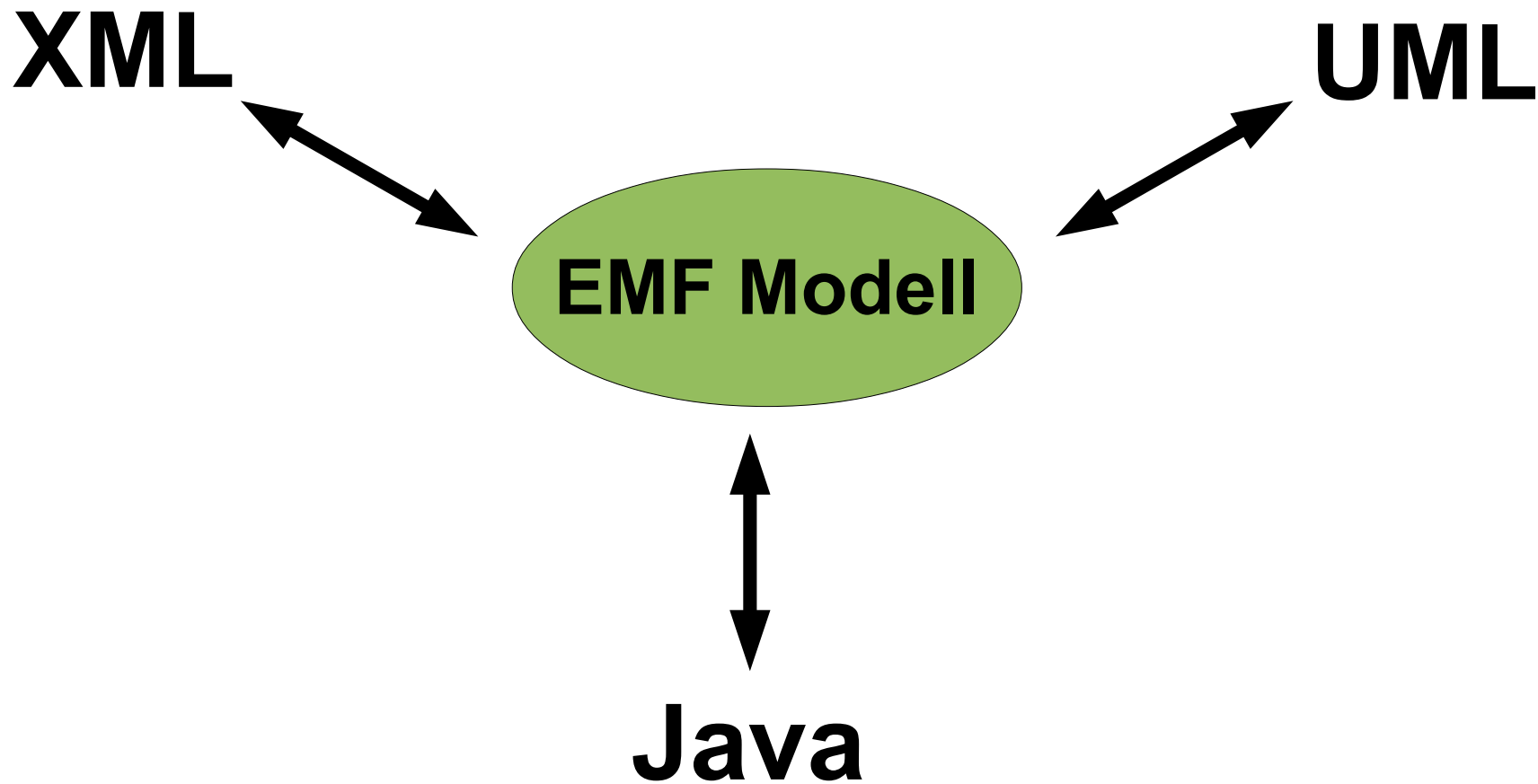
Standard MOF 2.4.2

- ISO/IEC 19508:2014
Information technology -
Meta Object Facility
(MOF)



Wie Metamodelle möglichst einfach in MDA-Tools umsetzen ?
→ Insbes. graphische Darstellung von Modellen auf Basis von Metamodellen, Codegenerierung aus Modellen.

Metamodelle aus Java-Klassen, UML-Diagrammen und XML-Dateien importierbar.



EMF.EMOF:

- Teil der MOF 2.0-Spezifikation (Essential MOF).

EMF.Ecore: Core EMF-Framework beinhaltet Meta-Model:

- **Um Modelle zu beschreiben.**
- **Laufzeitunterstützung** für Modelle inkl. Benachrichtigung bei Änderungen,
- **Persistenzunterstützung** durch Standard XML-Serialisierung,
- **API** um EMF-Modelle generisch zu verändern.

EMF.Edit:

- Generische und wiederverwendbare Klassen, um **Editoren** für EMF-Modelle zu erstellen.

EMF.Codegen:

- EMF Code-Generierungsframework: kann den für einen Editor für EMF-Modelle benötigten Code generieren.

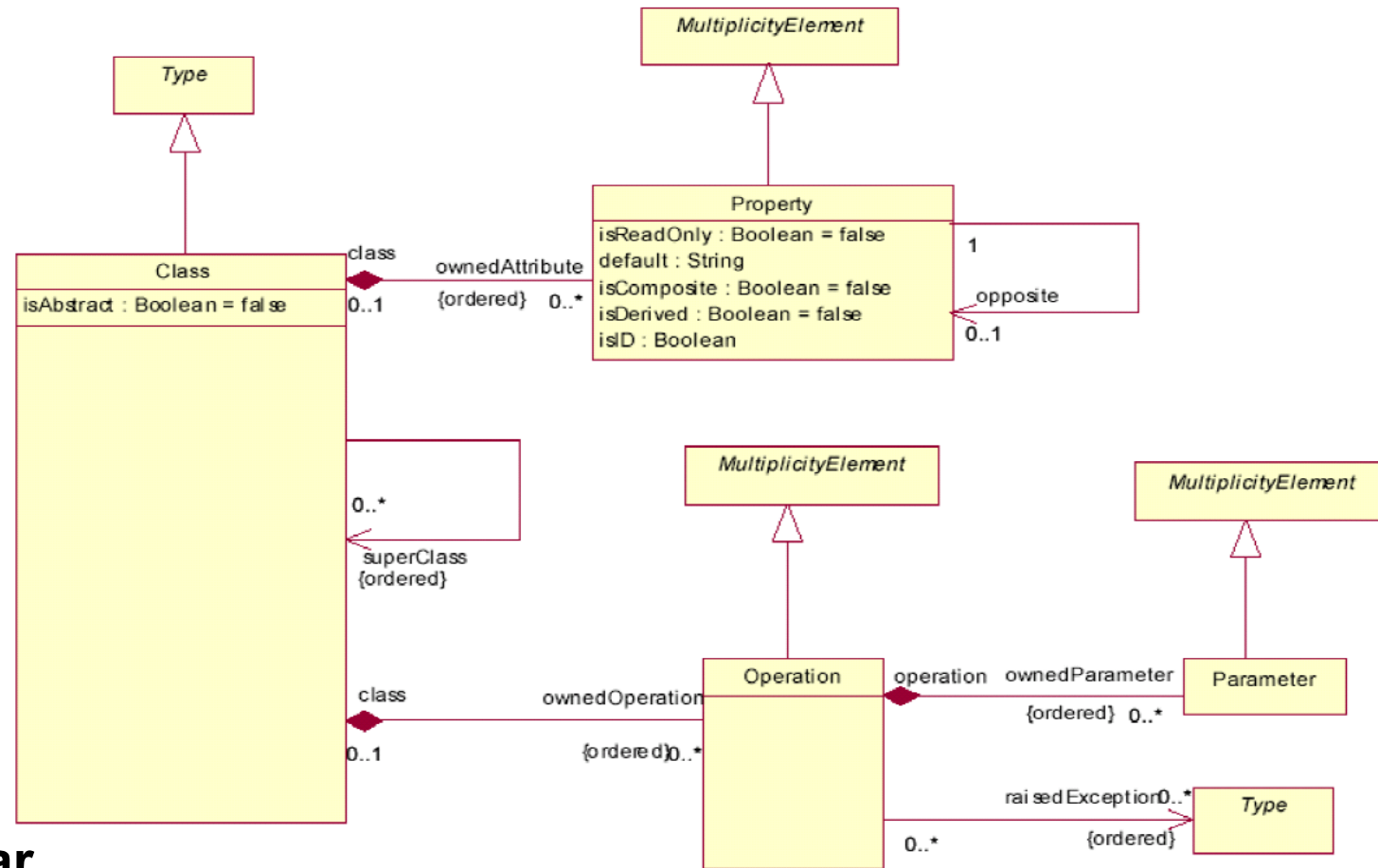
EMOF:

- Teil von MOF 2.0
- Zur **Definition von einfachen Metamodellen**.
- Nutzt OO-Konzepte.

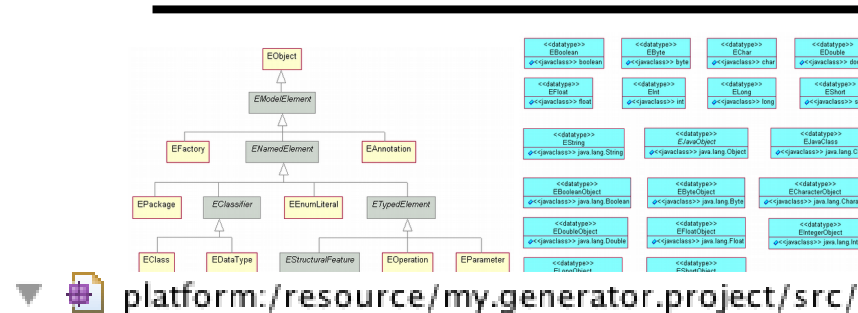
MOF 2.0 verwendet UML 2.0-Klassen-Diagramme.

- **Metamodell mit UML-Tools erstellbar.**
- **MOF 2.0 definiert Complete MOF (CMOF)** mit zusätzlichen Eigenschaften.

Beispiel: vereinfachtes Metamodell für Klassendiagramme (vgl. Teil 1.2 Folie 29 !)



Ecore



Metametamodel

„My Data
Models“

Concrete
„Data Model“

Concrete
Objects

```
platform:/resource/my.generator.project/src/metamodel/metamodel.ecore
└─ metamodel
   └─ Person bob = new Person();
      bob.setName("Bob");
      bob.setAge(35);
      Address address = new Address();
      address.setStreet("Mulholland Drive");
      address.setZip("8079");
      address.setCity("Los Angeles");
         Feature age
         Feature address
         Entity Address
            Entity Address
```

```
Person bob = new Person();
bob.setName("Bob");
bob.setAge(35);
Address address = new Address();
address.setStreet("Mulholland Drive");
address.setZip("8079");
address.setCity("Los Angeles");
```

/Model.xmi

Model Instance

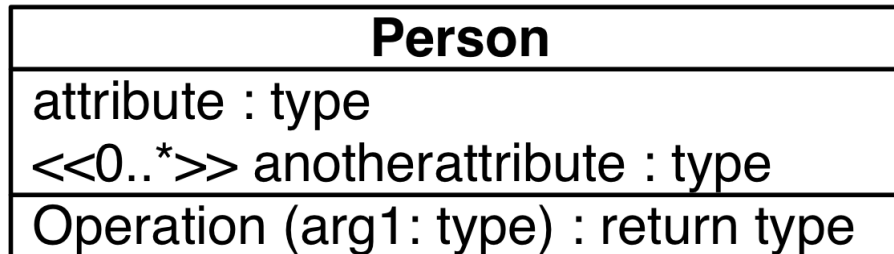
- “a modeling framework and code generation facility for building tools and other applications based on a structured data model” [EMF]
- part of the Eclipse Modeling Project [EMP]
- pragmatic bottom-up approach
- uses simple model specifications to generate:
 - corresponding Java classes
 - adapter classes for viewing and manipulating models
 - basic tree view editors
 - XML schemas

- EMF models are specified in XMI (XML Metadata Interchange)
 - OMG standard
 - interchange format for data whose metamodel is expressable in Meta-Object Facility (MOF)
 - EMF models contain:
 - simple object definitions
 - object attributes and operations
 - relationships between objects
 - simple conditions on objects and relationships (e.g. multiplicity)
- essentially: the class diagram portion of UML

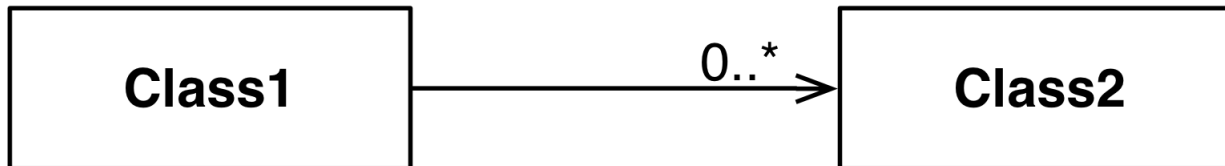
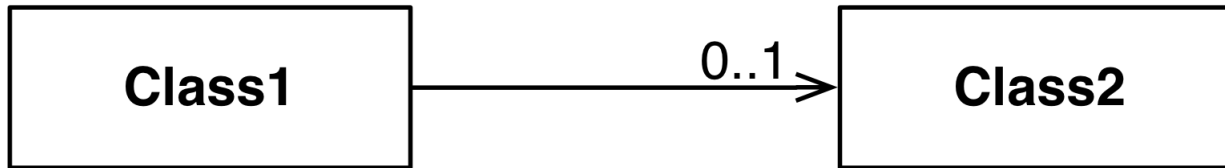
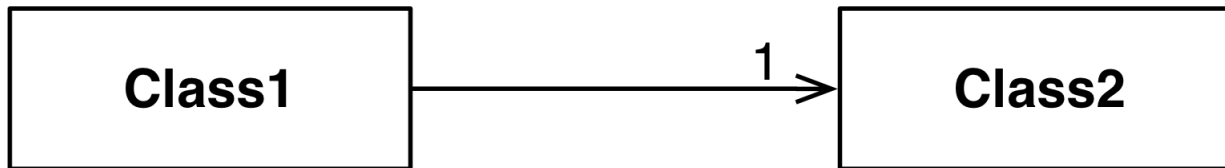
- classes, abstract classes and interfaces



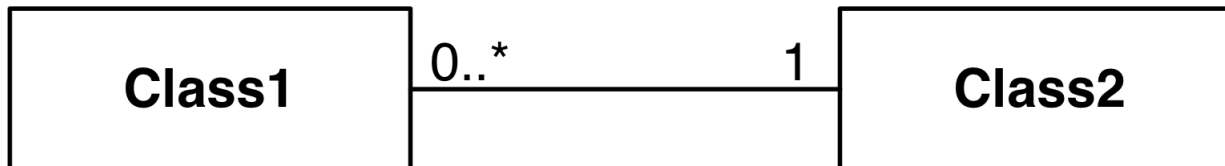
- operations and attributes



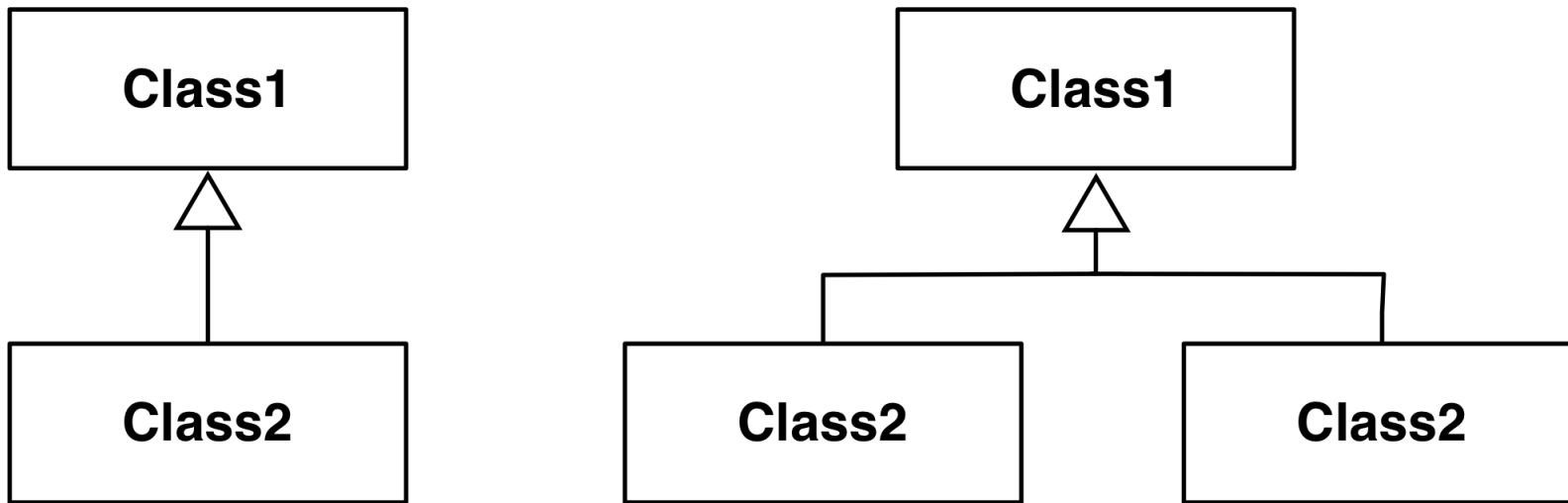
- one-way associations



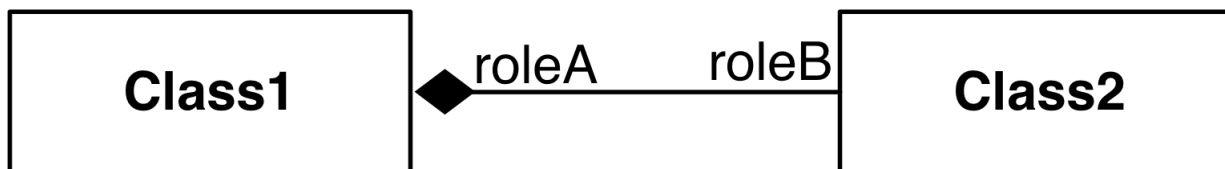
- bidirectional associations



- inheritance

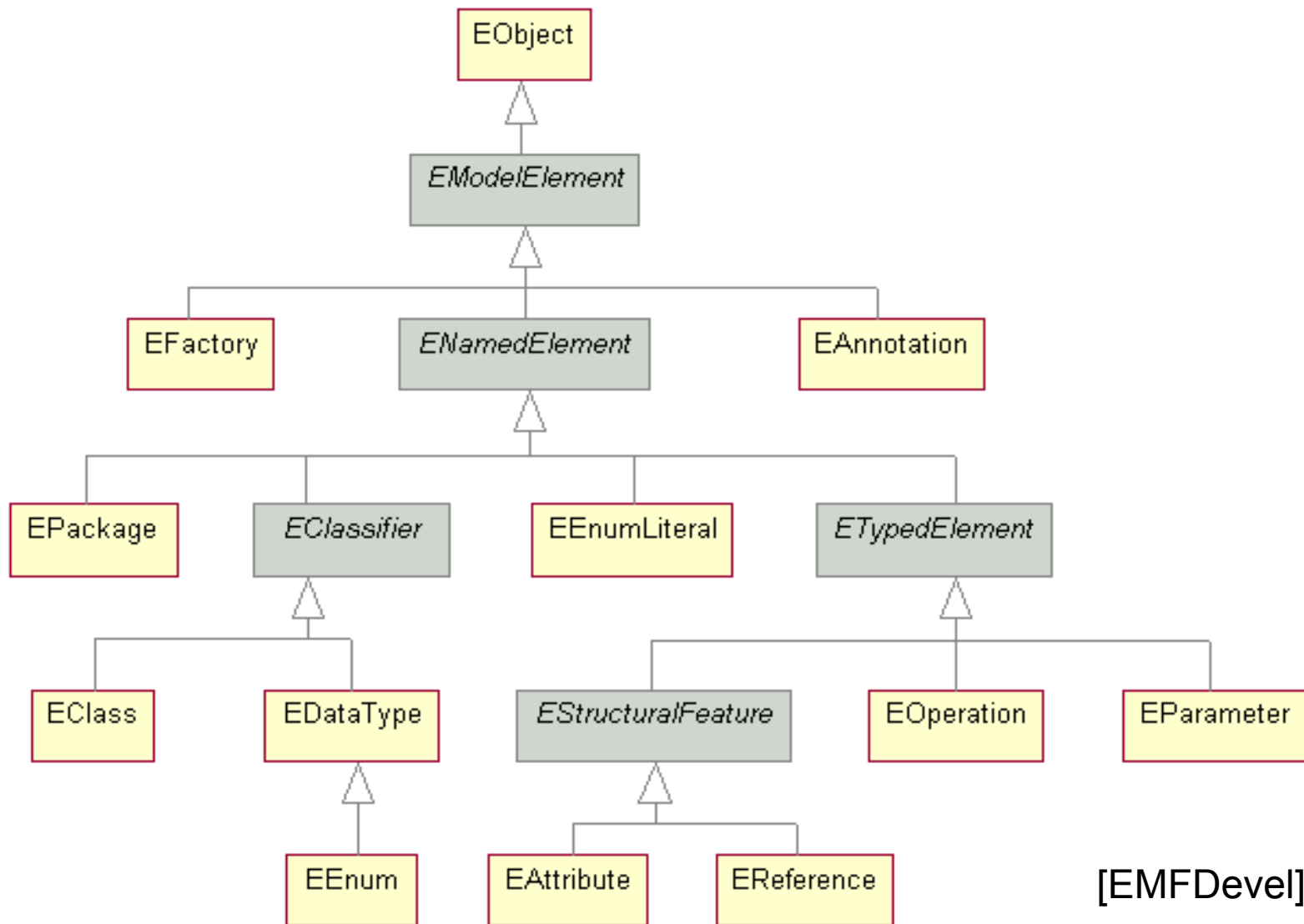


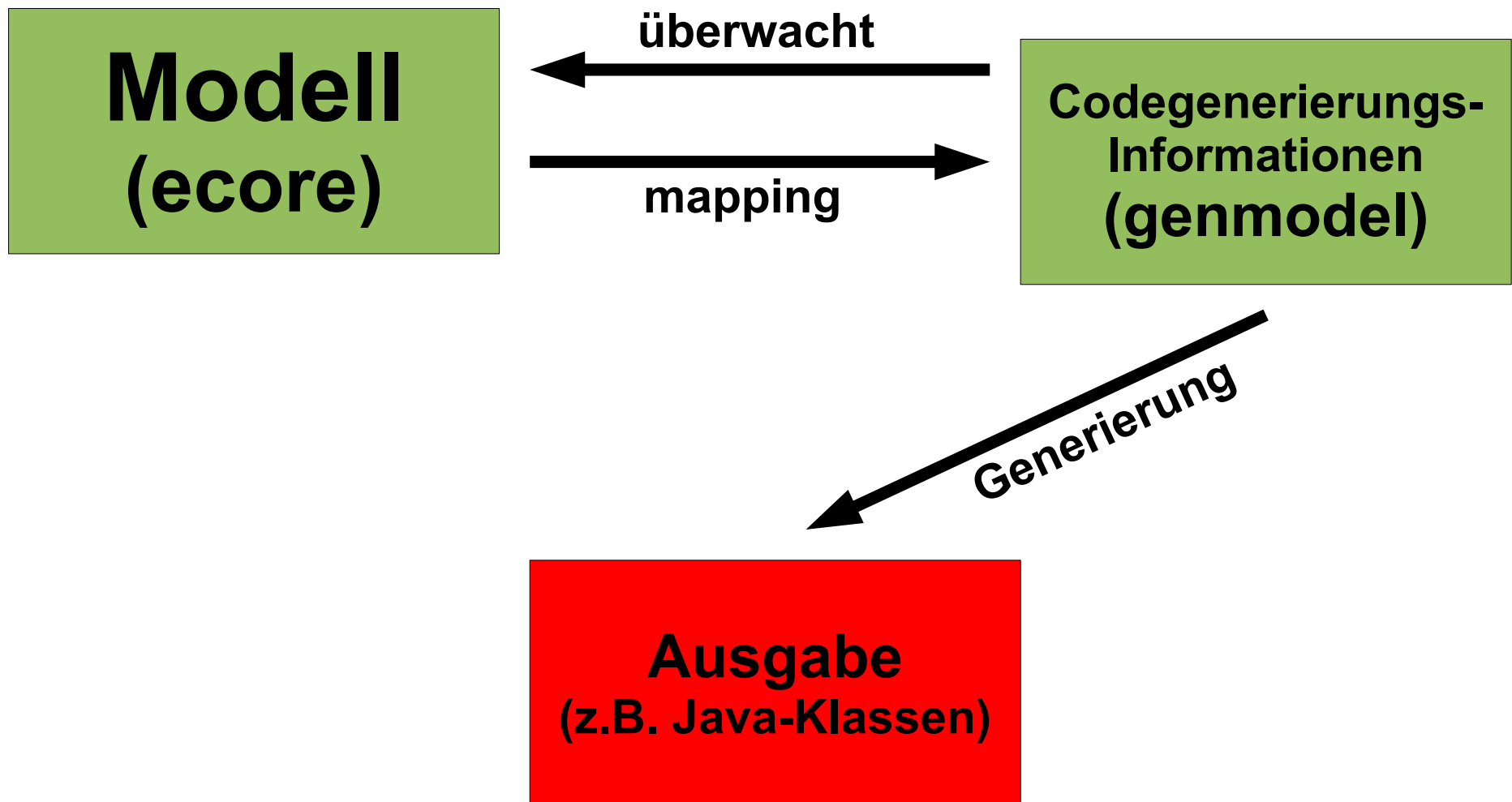
- containment associations



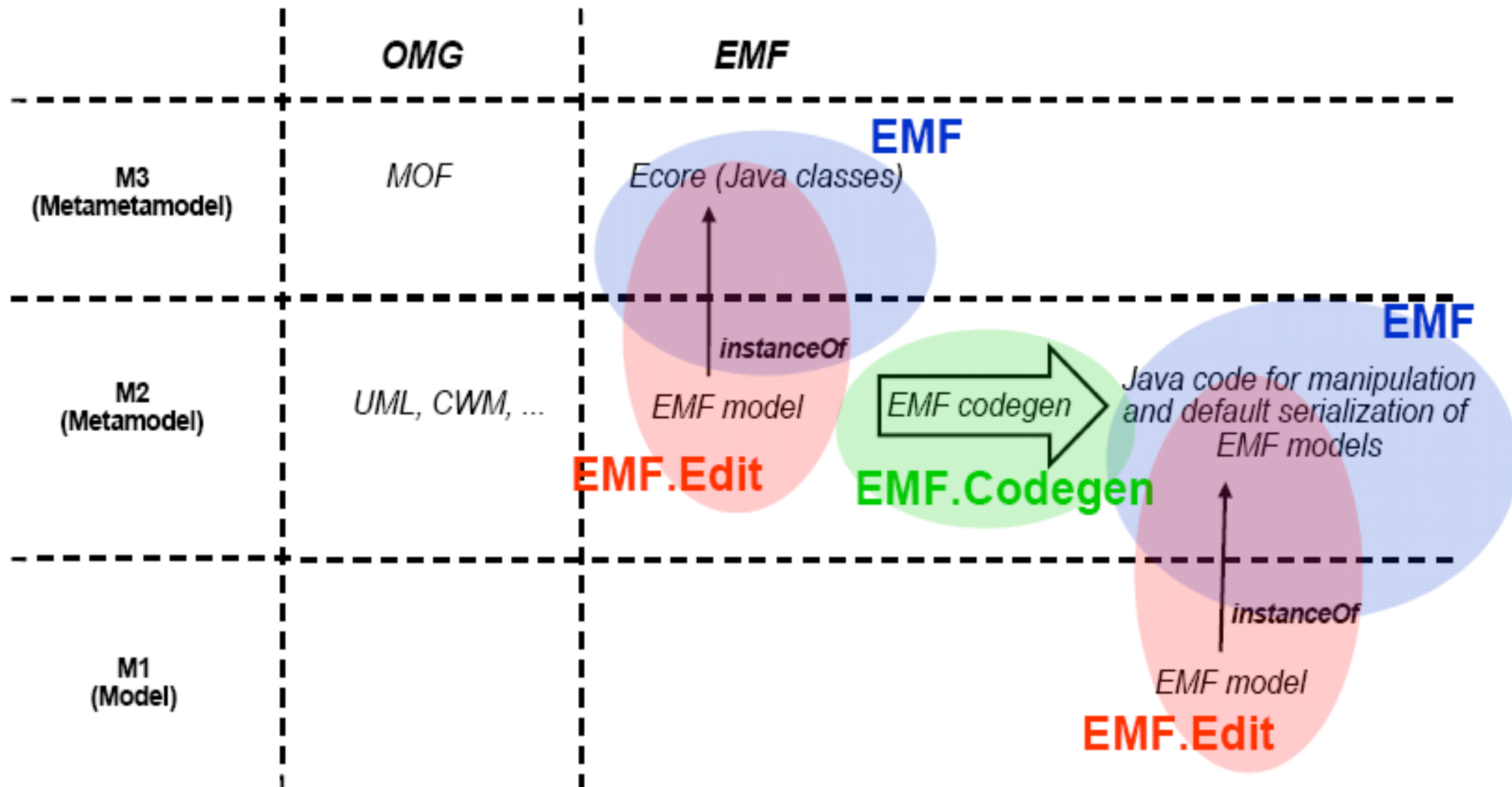
- and also enumerations and data types

- EMF model concepts are described by Ecore
- Ecore = EMF's metamodel
- very similar to OMG's EMOF (Essential Meta-Object Facility, a lightweight version of MOF)
- Ecore is reflexive, i.e. it can be used to describe itself
 - Ecore metamodel is an Ecore model
 - Ecore is just another EMF model
- stored as XMI (.ecore file)



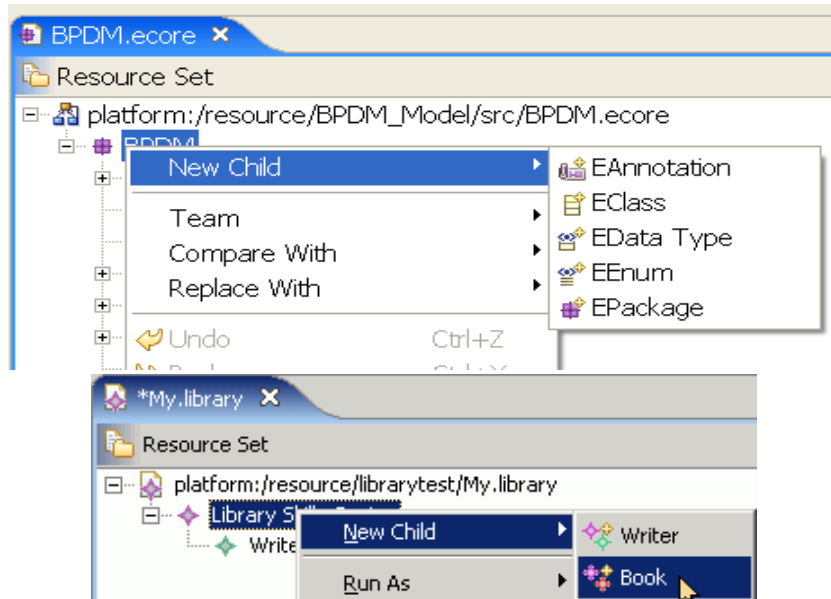


EMF – Überblick über Edit und Codegen

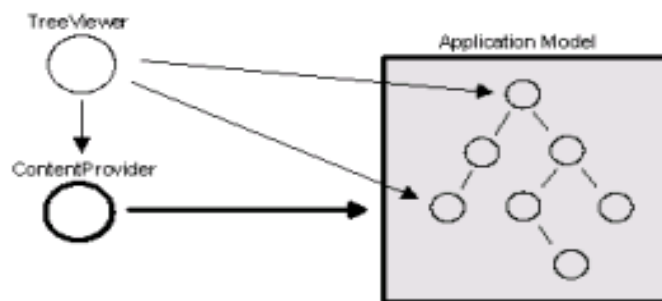


EMF.Edit

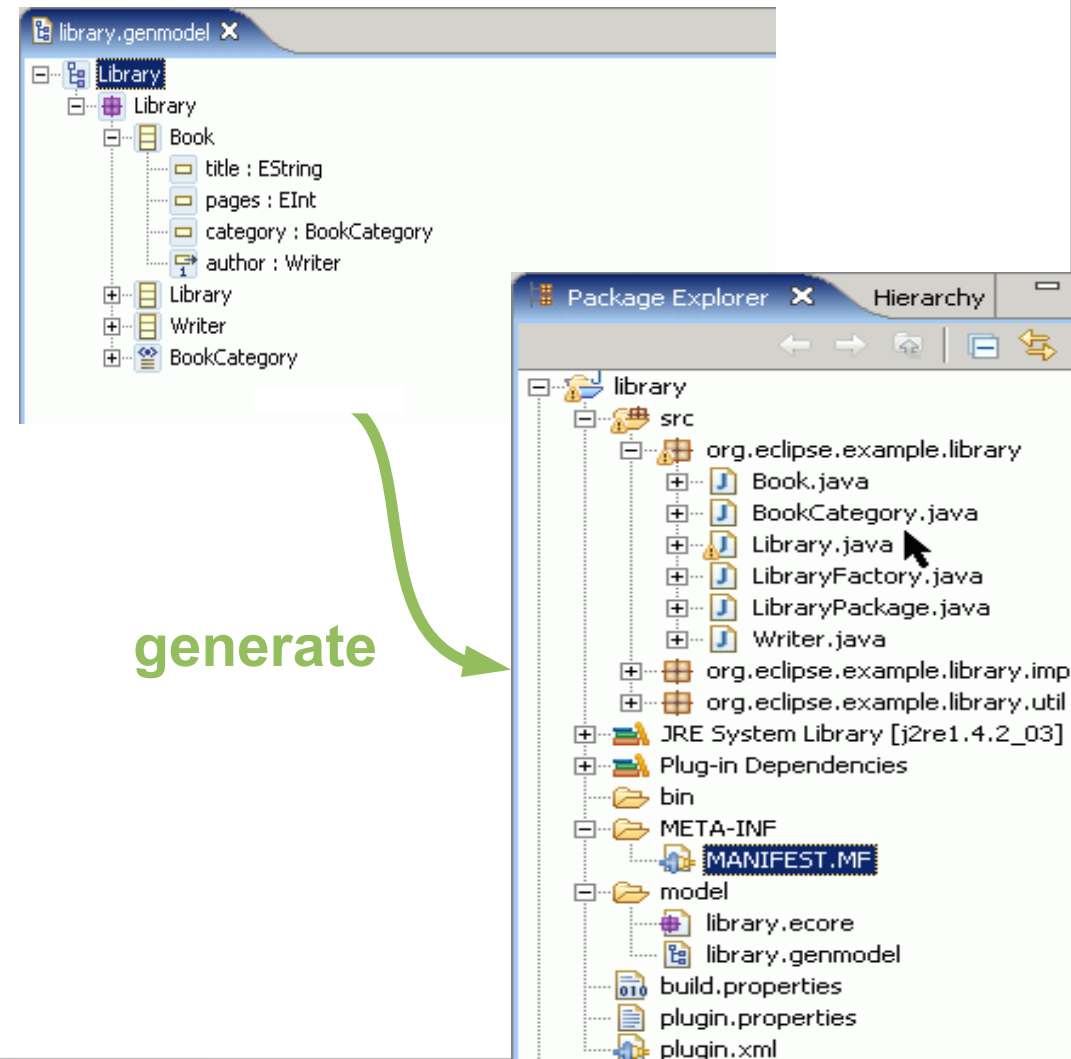
Modellierungseditor



**Content
Provider,
etc.**



EMF.Codegen



UML:

- EMF Ecore beschäftigt sich mit **Klassenmodellierungsaspekten** der UML.
- UML 2.0 Metamodell: In EMF Ecore implementiert.

MOF:

- Meta-Object Facility definiert konkrete Untermenge von UML.
→ Beschreibung der **Modellierungskonzepte** innerhalb Repository.
- Vergleichbar mit Ecore.
- Ecore vermeidet einige komplexe Elemente von MOF.
→ Fokus auf Tool-Integration als Management von Metadaten-Repositories.

XMI (XML Metadata Interchange):

- Zur **Serialisierung von Modellen**.
- Verwendung von EMF-Modell und Ecore selbst.

MDA:

- EMF unterstützt Hauptkonzept der MDA.
→ **Modelle für Entwicklung / Generierung** (nicht nur Dokumentation).

Welche **Aussagen** passen zu den angegebenen **Begriffen** ?

EMF.Emof

EMF-Framework; beinhaltet Meta-Model, um Modelle zu beschreiben.

EMF.Edit

Für **EMF-Modell-Editor** benötigten Code generieren.

EMF.Ecore

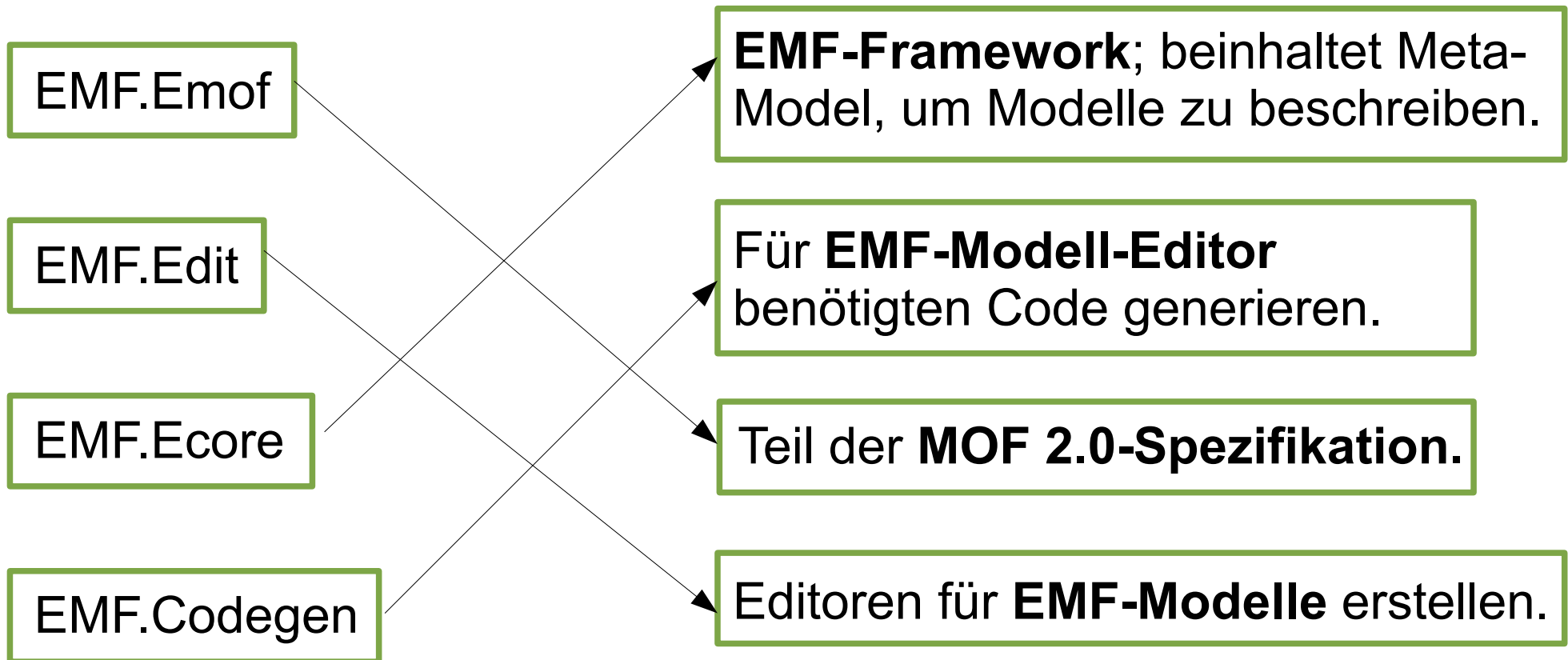
Teil der **MOF 2.0-Spezifikation**.

EMF.Codegen

Editoren für **EMF-Modelle** erstellen.

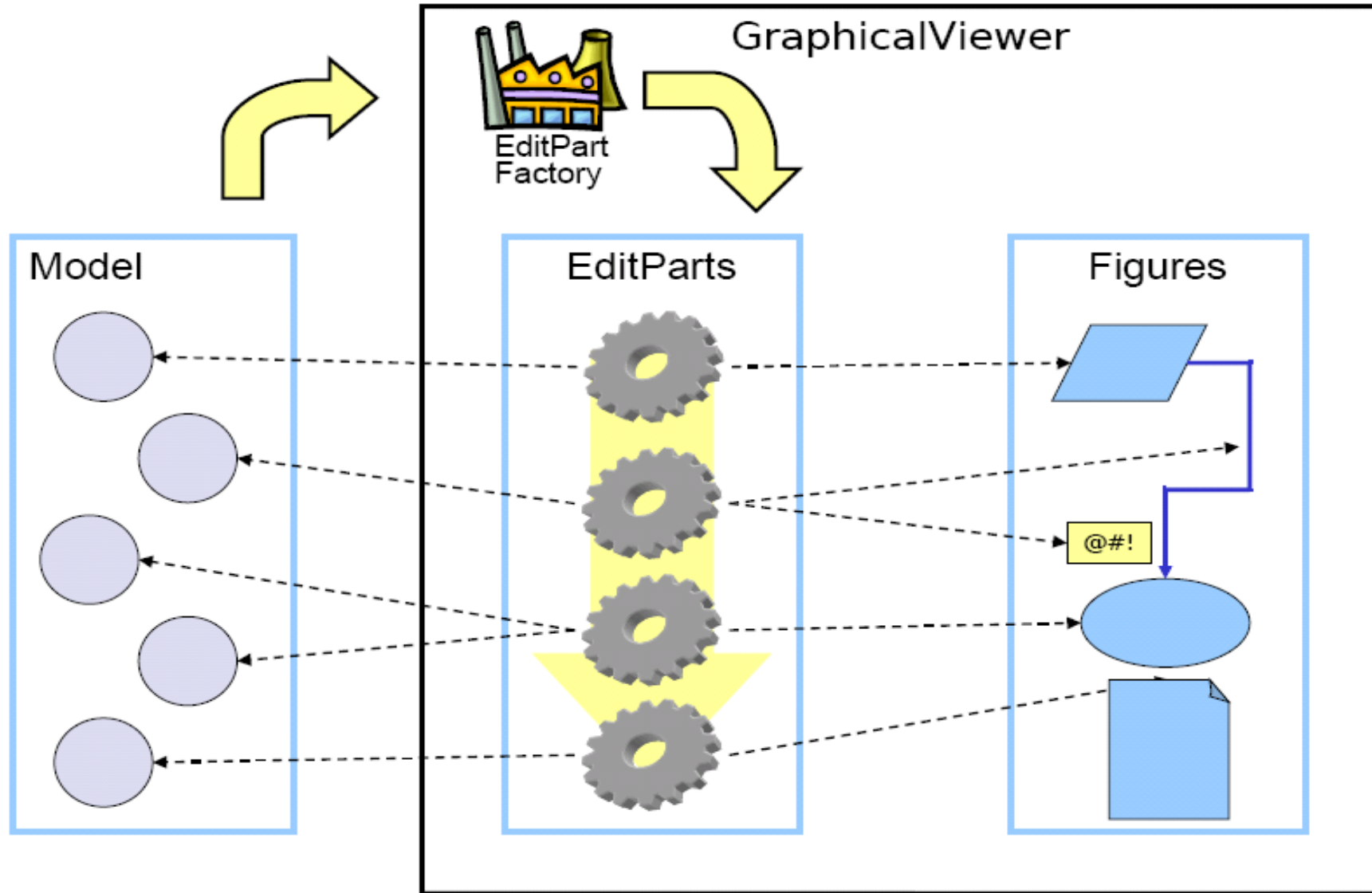
Diskussionsfrage: EMF-Bestandteile

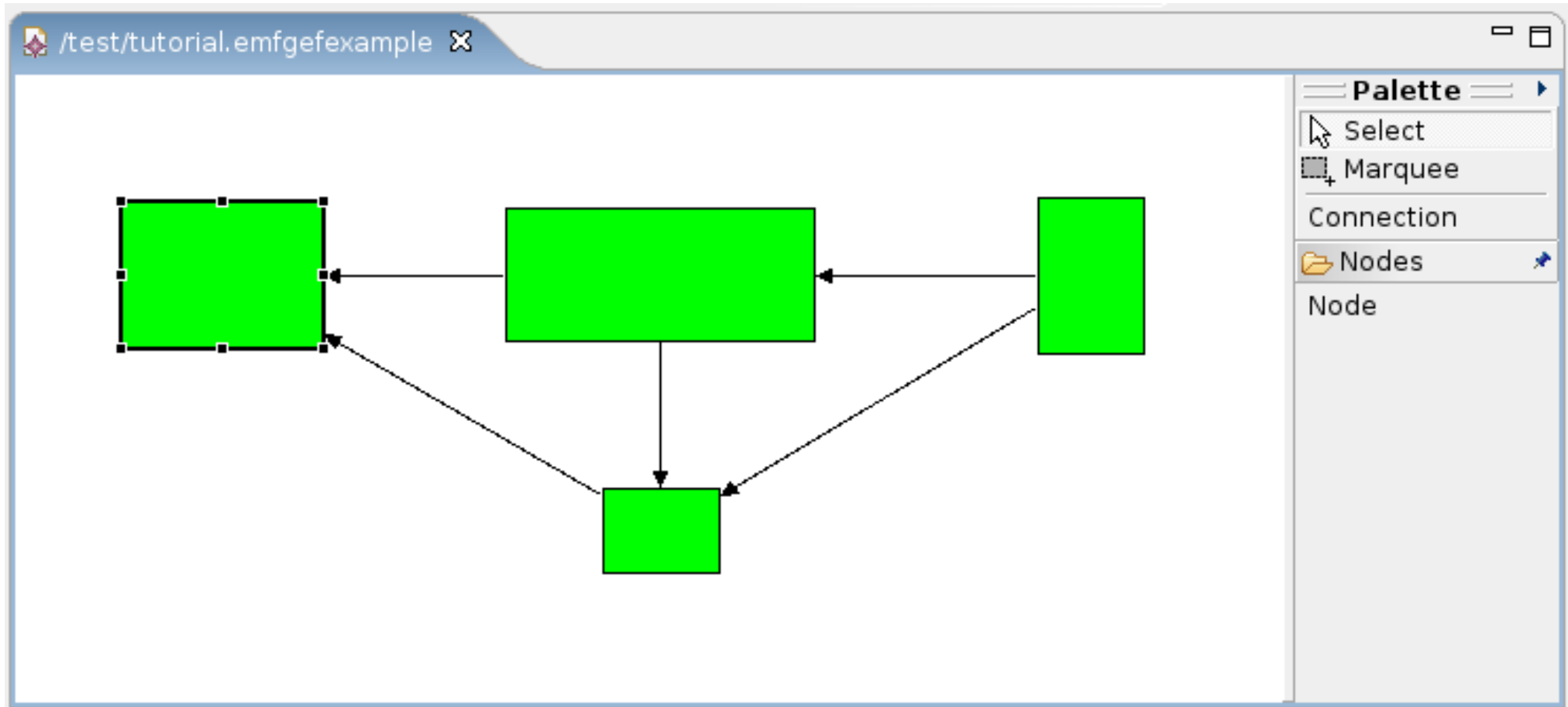
Welche **Aussagen** passen zu den angegebenen **Begriffen** ?



- Framework: **Modelle graphisch darstellen.**
- **Interaktion** mit Modell:
 - Verarbeitung von Benutzereingaben durch Maus und Tastatur.
 - Interpretation der Eingaben.
 - Möglichkeiten Modell zu verändern.
 - Änderungen rückgängig machbar (undo/redo).
- **Workbench Funktionen:**
 - Aktionen und Menüs.
 - Toolbars.
 - Keybindings.
- **Plugin** von Eclipse.
- Baut auf **Model-View-Controller Pattern** auf.
- **Ziel:** Wiederverwendete Funktionalitäten nicht jedesmal neu entwickeln.

MVC in GEF: EditPartFactory



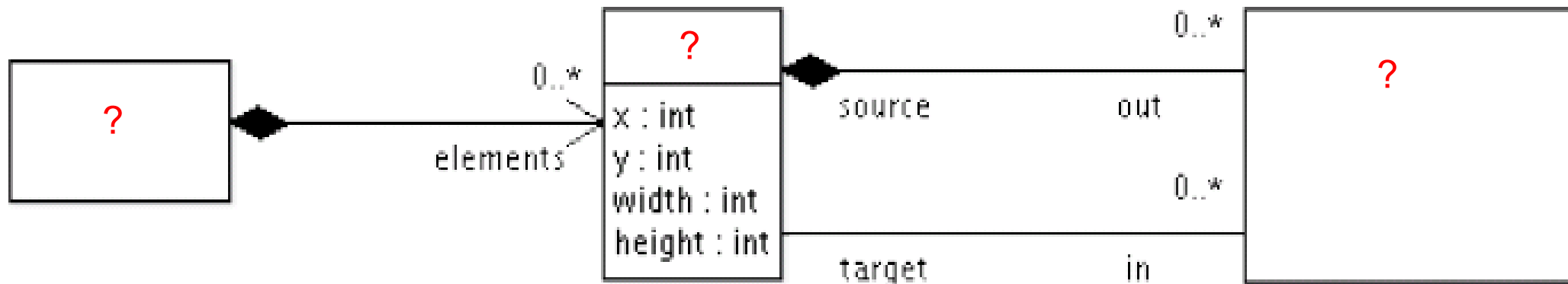


- **Erstellen eines neuen Diagramms** mittels Wizard.
- Öffnen eines existierenden Diagramms.
- **Speichern von Änderungen.**
 - auch als neues Dokument („speichern als“).
- **Palette mit Selektionstools** und Elementen.
- **Erstellen von Knoten** (*node*).
- Erstellen von Verbindungen (*connection*) zwischen Knoten.
- **Löschen von Knoten** und Verbindungen.
- Verschieben von Knoten.
- Ändern der Größe von Knoten.
- Alle **Veränderungen rückgängig machbar** (*undo*) und wiederherstellbar (*redo*).

Diskussion: Metamodell für Beispiel-Editor ?

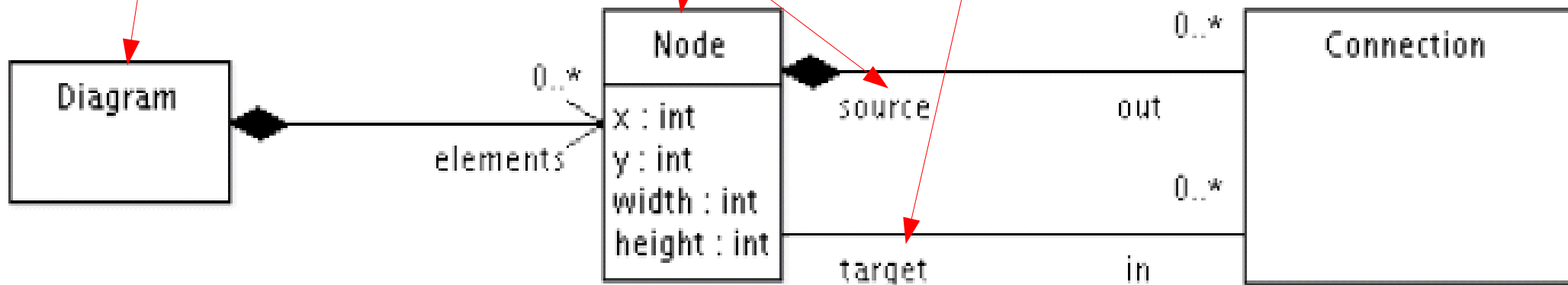
Was gehört an die
fehlenden Stellen im
unten abgebildeten
Metamodell ?

- Erstellen eines neuen **Diagramms** mittels Wizard.
- Öffnen eines existierenden Diagramms.
- Speichern von Änderungen.
 - auch als neues Dokument („speichern als“).
- Palette mit Selektionstools und Elementen.
- Erstellen von **Knoten** (*node*).
- Erstellen von **Verbindungen** (*connection*) zwischen Knoten.
- Löschen von **Knoten** und Verbindungen.
- Verschieben von Knoten.
- Ändern der Größe von Knoten.
- Alle Veränderungen rückgängig machbar (*undo*) und wiederherstellbar (*redo*).



Diskussion: Metamodell für Beispiel-Editor ?

- **Diagram:** Wurzelement.
- Diagramm enthält Knoten (*node*).
- Knoten besitzen **Quell- und Zielverbindungen** (*source Connection / target Connection*).



Vor- und Nachteile für die Verwendung von EMF mit GEF

Vorteile:

- **Kostengünstige Möglichkeit** für modellbasierte Softwareentwicklung.
- **Effektivität** durch automatische Konsistenzerhaltung der Modellrepräsentanten.
- **Mächtige Codegenerierung** erspart viel stupiden Programmieraufwand.

Nachteile:

- Modellierungssprachenschatz nicht mächtig wie UML (**Essential MOF**).
 - Aber meist ausreichend.

Eclipse Modeling Framework (EMF):

- Spezifische Realisierung der OMG MOF-Konzepte mit Eclipse und Java.
- Integriert im Eclipse Tools Projekt.

Textual Frameworks

- Xtext

Graphical Editing Framework (GEF):

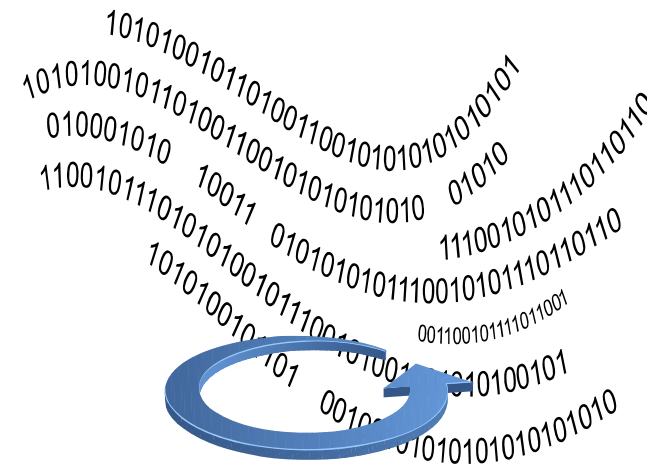
- Framework zur Darstellung von Modellen.
- Geschieht auf Basis eines EMF-Metamodells oder eigenständig.

Graphical Modeling Framework (GMF):

- Versuch, EMF und GEF zu integrieren.
- Graphical Tooling Infrastructure (Graphiti)

Metaprogrammierung

- Metaprogrammierung bezeichnet Möglichkeit von Programmen sich selbst zu analysieren und zu modifizieren
- Reflection ähnlich aber Struktur bleibt statisch
- Erlaubt adaptive Systeme entweder zur Lauf- oder Entwicklungszeit
- Unterscheidung
 - Objektsprache
 - Metasprache



Nach Bearbeitungszeitpunkt:

- statisch (zur Compiler-Zeit)
- dynamisch (zur Laufzeit)

Nach Sprachen:

- homogen (Metasprache ist Objektsprache)
- heterogen (sonst)

Nach Stufen:

- mehrstufig (Objektsprache ist selbst Metasprache)
- einstufig (sonst)

Beispiele, Anwendungen und Produkte welche Metaprogrammierung verwenden?

- Interpreter—analysiert zu interpretierendes Programm
- Compiler—analysiert Quellprogramm, transformiert Zwischencode und generiert Zielprogramm
- Scanner- und Parsergeneratoren - Objektprogramm ist selbst wieder ein Metaprogramm (mehrstufige Metaprogrammierung)
- Makroprozessoren
- Generatoren für effiziente Datenbankoperationen
- Scaffolding Tools

Drei unterschiedliche Wege

- Offenlegung der Interna von Runtime-Engine via APIs
 - Programm modifiziert sich via API
- Dynamische Ausführung von Kommandos (im Ausführungskontext) z.B. als Programm als Zeichenkette
- Programm-Transformationssprachen (Mehrzweck), z.B.:
 - Compiler für beliebige Objektsprachen
 - Template Metaprogrammierung

Repräsentation von Objektprogrammen

String: “let x = 3 in x+1”

Datentyp: Let(“x“, CI 3) (Var “x“ :+: CI 1)

Abstrakter Syntaxbaum(!)

Problem: Syntaxfehler schwer auffindbar

Phasen der Codegenerierung heißen Stages

Multi-Staging = Code erzeugt wieder Code

In Template-Metaprogrammierung verwendet Compiler Templates und generiert Zwischencode, welcher dann mit Rest des Programmcodes compiliert wird.

Art von Compilezeit-Ausführung (z.B. templates in C++)

Schritte:

- Definition von Templates
 - definiert generische Form des Programmcodes
- Instantiierung der Templates
 - generierte spezifische Form des Programmcodes


```
unsigned int factorial(unsigned int n) {  
    return n == 0 ? 1 : n * factorial(n - 1);  
}
```

Aufruf von `factorial(0)` liefert 1.
Aufruf von `factorial(4)` liefert 24.

C++ Code der Fakultät
von n berechnet

```
template <int n>  
struct factorial {  
    enum { value = n * factorial<n - 1>::value };  
};
```

```
template <>  
struct factorial<0> {  
    enum { value = 1 };  
};
```

Aufruf von `factorial<0>::value` liefert 1.
Aufruf von `factorial<4>::value` liefert 24.

C++ Code mit Templates
der Fakultät von n
berechnet

- Compile-Zeit vs. Ausführungszeit
 - Compile-Zeit länger durch Compile-Zeit-Ausführung
- Generische Programmierung
 - Fokus auf Architektur und nicht Programmcode
 - Sehr gute Codeminimierung und Wartbarkeit erreichbar(!)
- Lesbarkeit des Quelltexts?
- Probleme bei Fehlersuche/Debugging
 - Fehlersuche in automatisch generiertem Code
 - Nachverfolgbarkeit (Traceability) nicht unterstützt

- Metamodelle definieren Modelle
- EMF ist Metamodellierungssprache
- Metamodellierung von UML durch MOF
- Metaprogrammierung bezeichnet Möglichkeit von Programmen sich selbst zu analysieren und zu modifizieren.

In diesem Abschnitt: Eclipse Modeling Framework (EMF)

- Technische Grundlagen für UML-Werkzeuge und MDA.
- Metamodellierung und -programmierung

Damit Ende des Kapitel 1: Modellbasierte Entwicklung.

Als nächstes: **Softwarequalitätsmanagement** und insbesondere **Softwareverifikation**.

Insbesondere unter Verwendung von Techniken aus Kap. 1
(**Testautomatisierung** durch **Modellbasiertes Testen mit UML**,
Einhaltung von **Constraints mittels OCL**).

Anhang

(weitere Informationen zu Nachbereitung)

Softwarekonstruktion
WS 2016/17



Modellierungsframework und Tool zur **Code-Generierung** basierend auf strukturiertem Datenmodell.

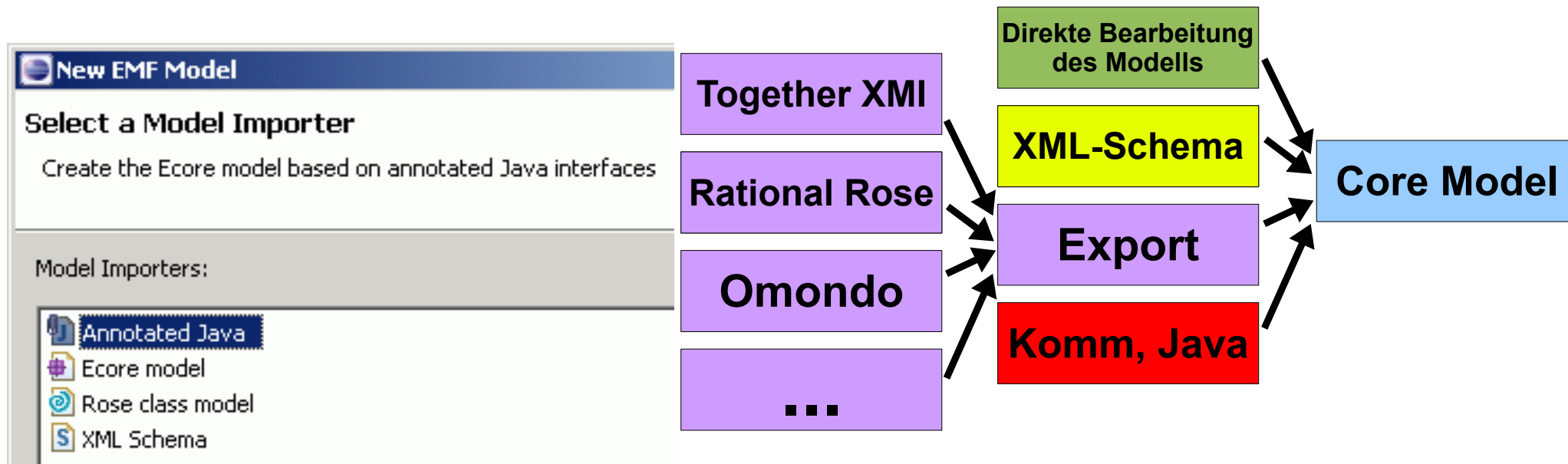
Ausgehend von Modellspezifikation in XMI bietet EMF:

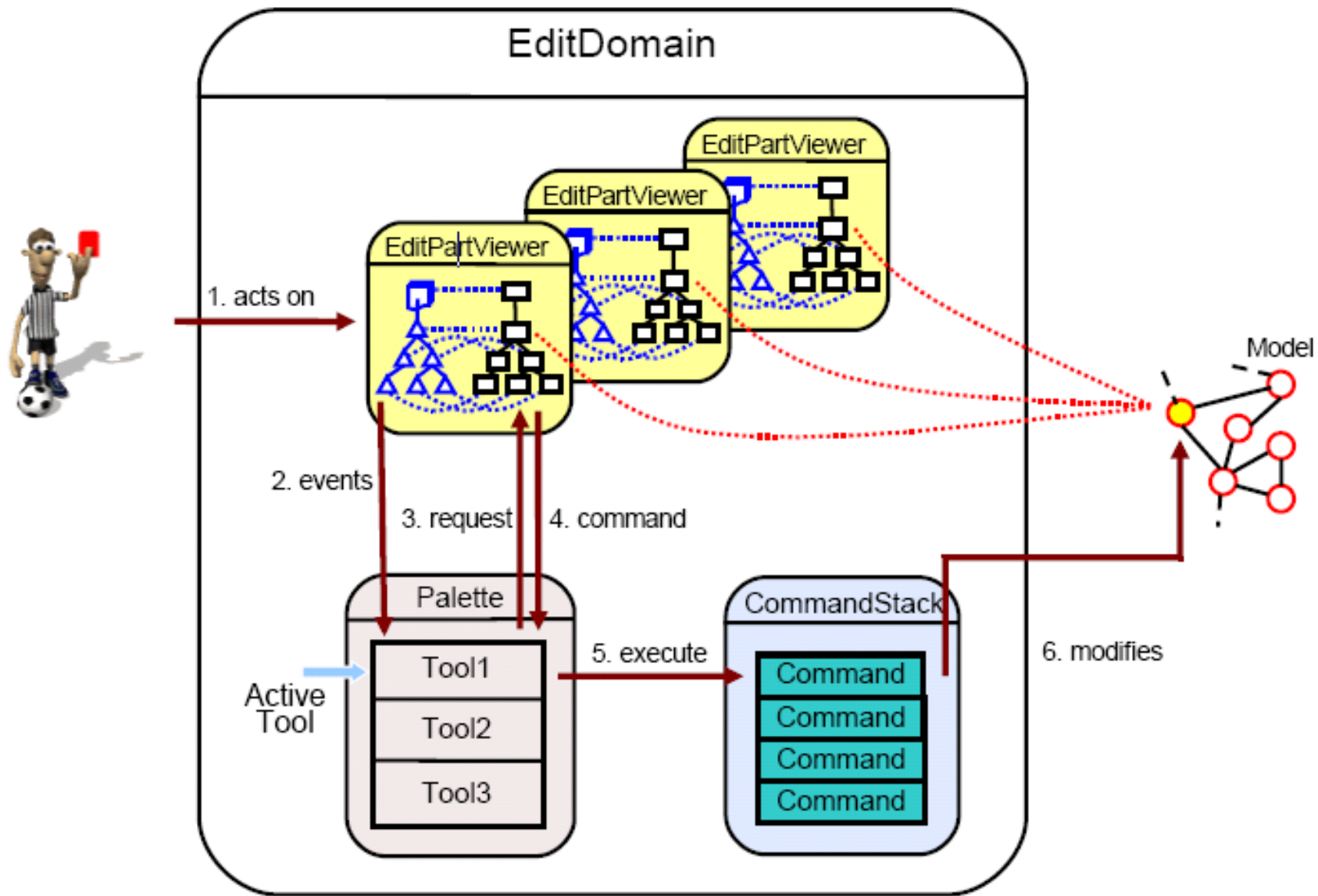
- **Tools und Laufzeitunterstützung.**
→ Javaklassen aus Modell erstellen.
- **Adapterklassen:** Einfache Sicht und kommandobasiertes Editieren des Modells.
- Grundlegender **Editor.**

Grundlage für **Interoperabilität** zwischen EMF-basierten Anwendungen.

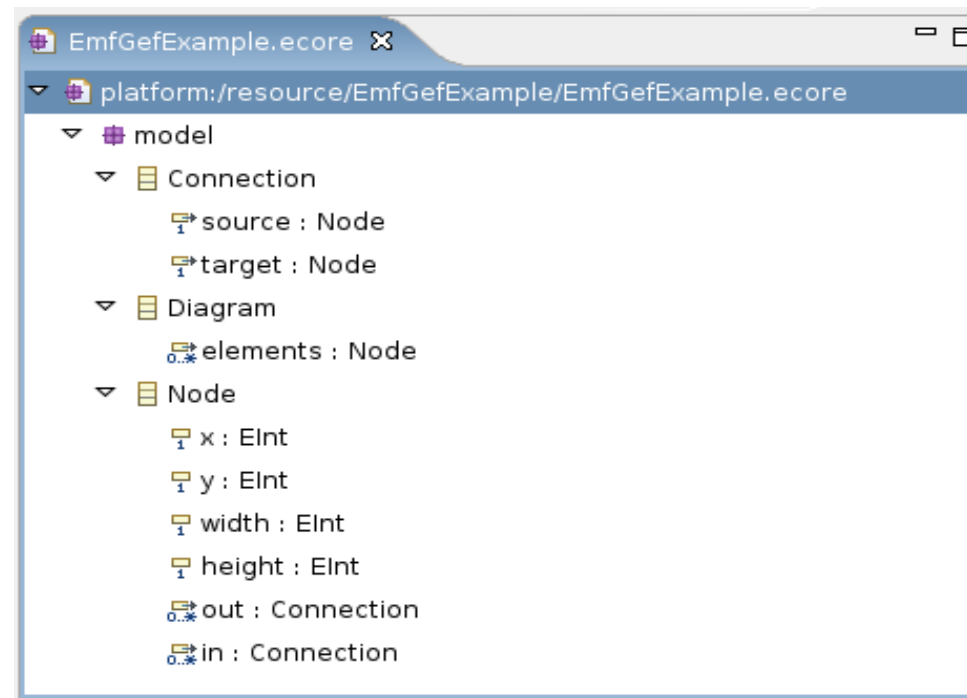
EMF (**Meta-**)Modelle wie folgt erstellbar:

- XMI-Datei **direkt im Texteditor** erstellen (→ Ecore model).
- Verwendung eines **Modellierungstools** wie bspw. Rational Rose und Export als XMI-Dokument (→ Rose class model).
- Annotierte Java-Klassen und **Interfaces einlesen** (→ Annotated Java).
- **XML-Schema verwenden**: Modell-Serialisierung beschreiben (→ XML Schema)





- **Erstellt mit ArgoUML** (<http://argouml.tigris.org>).
- **Export** von ArgoUML als **XMI**.
- **Transformation von ArgoUML XMI** nach Ecore XMI mithilfe des Tools *argo2ecore* (<http://argo2ecore.sourceforge.net>)



Zusätzliche Informationen um Java Klassen zu erstellen.

Allgemeine Informationen:

- Copyright.
- Name des Modells.
- ID des Plugins.

Einstellungen für EMF.Edit:

- Unterstützung zur **Erstellung von Kindelementen** durch Commands.
- Icons.
- Plug-in Klassen.

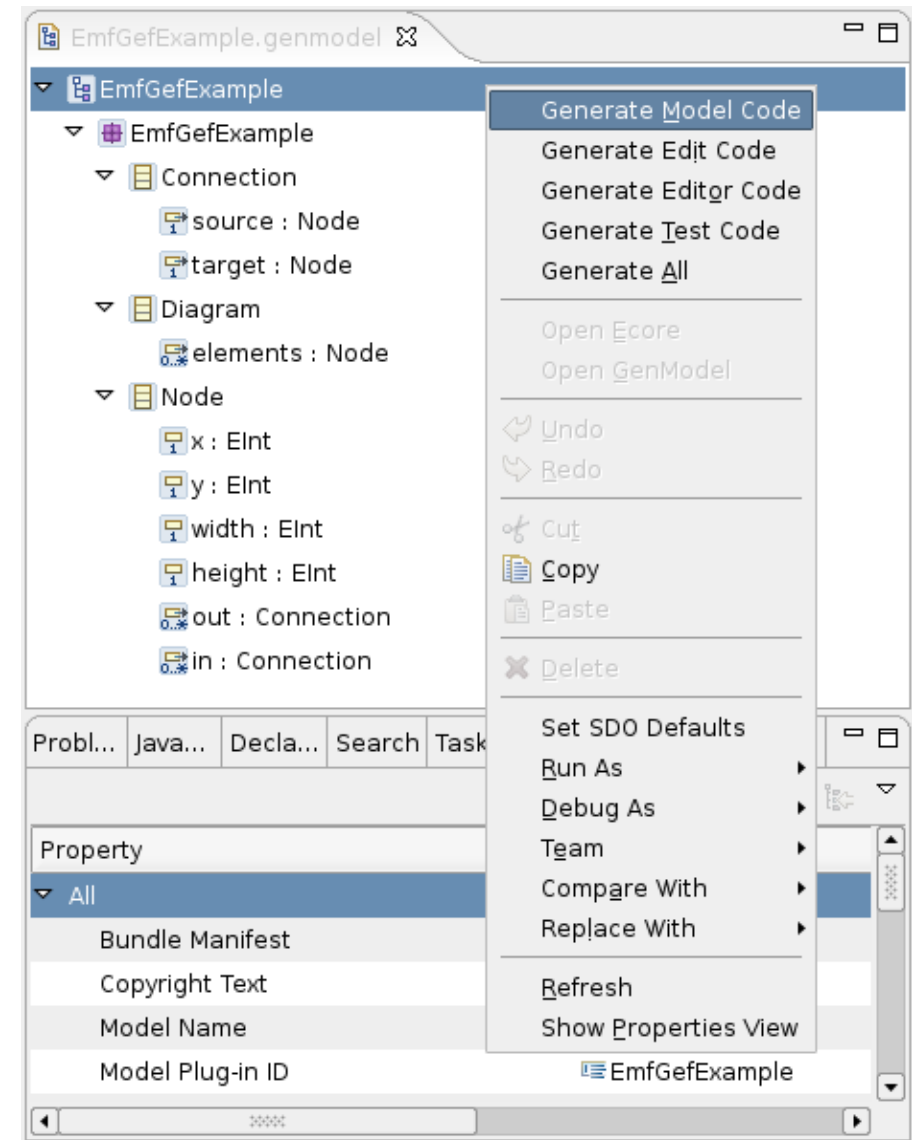
Einstellungen für EMF Editor.

Template & Merge:

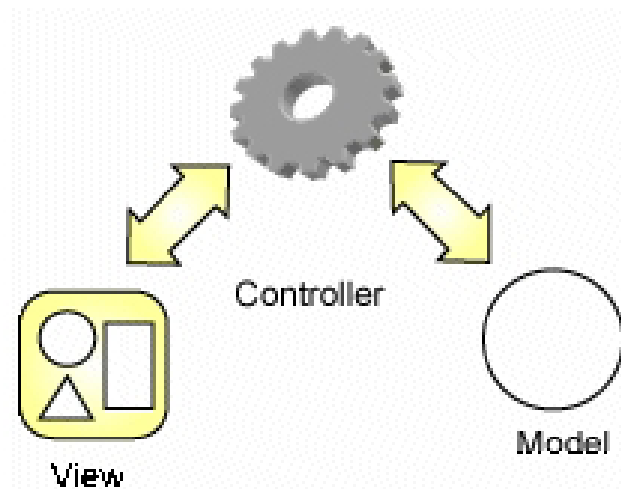
- Automatische Formatierung des Codes.
- **Dynamische Templates:** Java Klassen mithilfe von JET erzeugen.
→ Bei Bedarf anpassbar.

Einstellungen zur Property View.

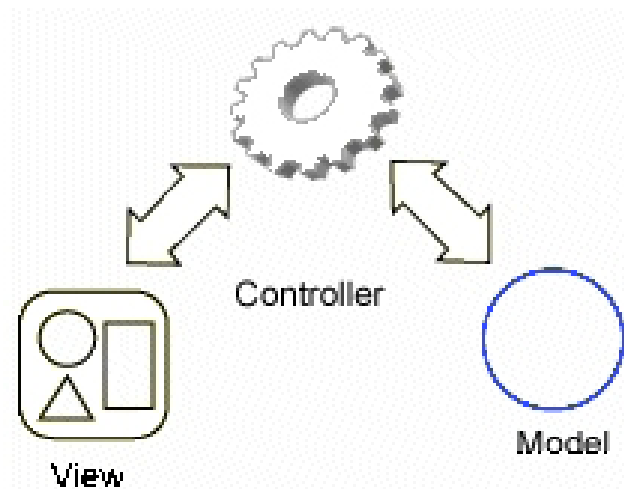
- **Erstellen des GenModels** aus Ecore Modell.
- **Erstellen des Modells** aus GenModel.
- Wenn man Editor generiert, dann hat man an dieser Stelle einen **Baumeditor**, mit dem man Modell bearbeiten kann.



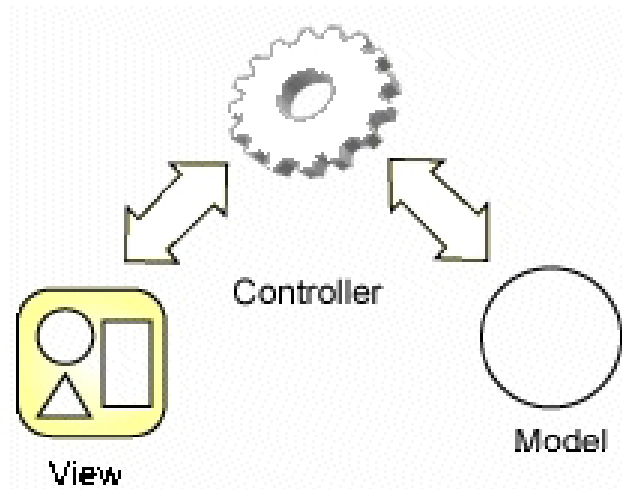
- **3 Schichten Modell.**
- **Strikte Trennung der Schichten.**
- Daten in **Modellschicht.**
- Visualisierung der Daten in **Viewschicht.**
- Kommunikation zwischen 2 Schichten in **Controllerschicht.**



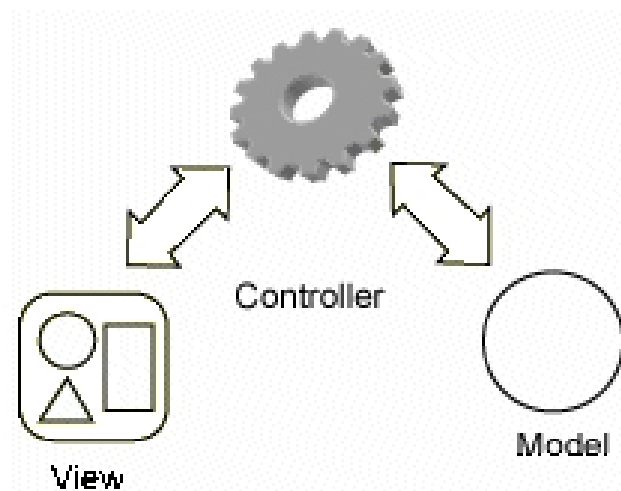
- Alle **persistente und wichtige Daten** ausschließlich hier gespeichert.
- **Container für Daten.**
- Kennt keine anderen Teile des Programms.
- Teilt **Änderungen** an sich mit über Listener.



- **Keine Daten** in Viewschicht.
- **Keine Modelllogik.**
- Kennt keine anderen Teile des Programms.
- **Abbildung der Daten** der Modellschicht.



- **Verbindung** von Modell- und Viewschicht.
- Leitet **Kommunikation** vom Modell an View weiter.
- **In GEF:** Unterklasse von EditPart.
- Zu jedem EditPart genau **ein Modell** und **genau eine View**.



Diskussionsfrage: Verwendung von MVC-Pattern

Welche konkreten Vor- und Nachteile bietet die Verwendung von MVC-Pattern **im Kontext von GEF** ?

Antwort:

Vorteile:

- Durch Change-update-Mechanismus ist das Model in allen Views immer aktuell visualisiert.

Nachteil:

- Für dasselbe Model sind mehrere View-Controller-Paare vorzusehen.
- Falls sich die Daten sehr oft und schnell ändern, kann es sein, dass das View die Veränderungen nicht schnell genug anzeigen kann.

- **GEF Beispiele (im Plug-in enthalten):**
 - Shapes (Einfachstes Beispiel).
 - Logic (Sehr umfangreiches Beispiel).
- **GEF Dokumentation:** <http://www.eclipse.org/gef/reference/articles.html>
- **GefDescription:** <http://eclipsewiki.editme.com/GefDescription>
- **EMF Dokumentation:** <http://www.eclipse.org/emf/docs.php>
- **EMF Übersicht:** <http://www.eclipse.org/emf/docs.php?doc=references/overview/EMF.html>
- **EMF.Edit Übersicht:** <http://www.eclipse.org/emf/docs.php?doc=references/overview/EMF.Edit.html>
- **EMF Book: Eclipse Modeling Framework (Overview and Developer's Guide):**
<http://www.awprofessional.com/content/images/0131425420/samplechapter/budinskych02.pdf>
- **Create an Eclipse-based application using the GEF:**
<http://www-128.ibm.com/developerworks/opensource/library/os-gef>
- **Using GEF with EMF:** <http://www.eclipse.org/articles/Article-GEF-EMF/gef-emf.html>
- **IBM Redbook: Eclipse Development using the Graphical Editing Framework and the Eclipse Modeling Framework:** <http://www.redbooks.ibm.com/abstracts/sg246302.html>

- Hudson, Randy; Shah, Pratik: *Tutorial #23 / GEF In Depth*;
<http://www.eclipse.org/gef/reference/GEF%20Tutorial%202005.ppt>
- EclipseCon 2005 und 2006: Vorträge zu EMF und GEF
- **Beispiele und Tutorials** von Eclipse EMF und GEF

1.3 Eclipse Modeling Framework (EMF)

Agenda

- Motivation und Überblick
- Eclipse Modeling Framework (EMF)
 - EMF-Modellimport
 - **EMOF und Ecore**
 - EMF Edit & Codegen
- Graphical Editing Framework (GEF)
 - Model-View-Controller (MVC)-Pattern
 - MVC in GEF
 - Weitere Konstrukte: EditPolicies und Commands
- Nutzung von EMF in GEF
 - Einführung eines Beispiels
 - Beispiel in EMF und GEF
- Zusammenfassung und Ausblick

- **Zu jeder Klasse** im Ecore Metamodell:
 - Java Interface.
 - Implementierung im Unterpackage impl.
- **Zu jedem Package:**
 - Eine Package Klasse.
 - Informationen zu Features und Metadaten des Modell.
 - **Factory Klasse:** Bietet Methoden zum Erzeugen neuer Objekte.

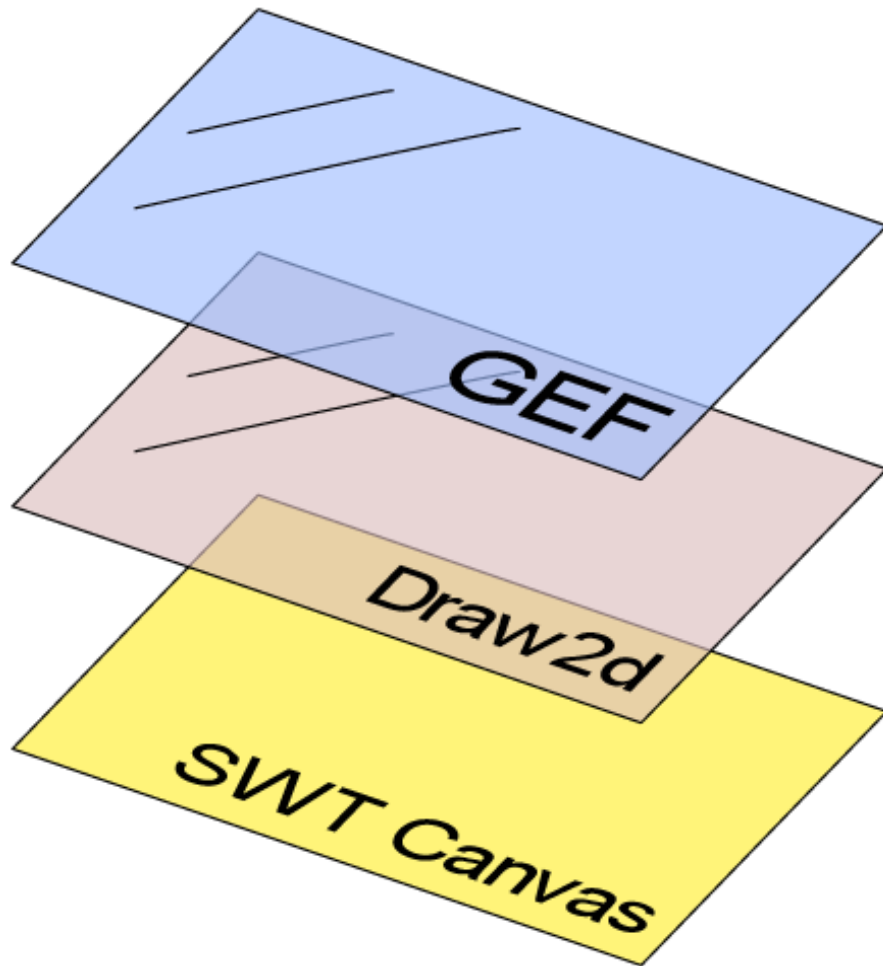
- **Edit Provider für jede Klasse** im Ecore Metamodell:
 - Informationen zu Kindern und Eltern vom Objekt.
 - Descriptoren zur Erzeugung von Kindern.
 - Commands zur Änderung des Objekts.
 - Informationen zur Erzeugung eines Baumes, der das Modell repräsentiert.
 - Text und Icon zum Objekt.
 - Informationen für Property Sheet.
- **Adapter Factory:**
 - Liefert richtigen Provider zum Objekt.

1.3 Eclipse Modeling Framework (EMF)

Agenda

- Motivation und Überblick
- Eclipse Modeling Framework (EMF)
 - EMF-Modellimport
 - EMOF und Ecore
 - EMF Edit & Codegen
- Graphical Editing Framework (GEF)
 - Model-View-Controller (MVC)-Pattern
 - MVC in GEF
 - Weitere Konstrukte: EditPolicies und Commands
- Nutzung von EMF in GEF
 - Einführung eines Beispiels
 - Beispiel in EMF und GEF
- Zusammenfassung und Ausblick

Was ist GEF?

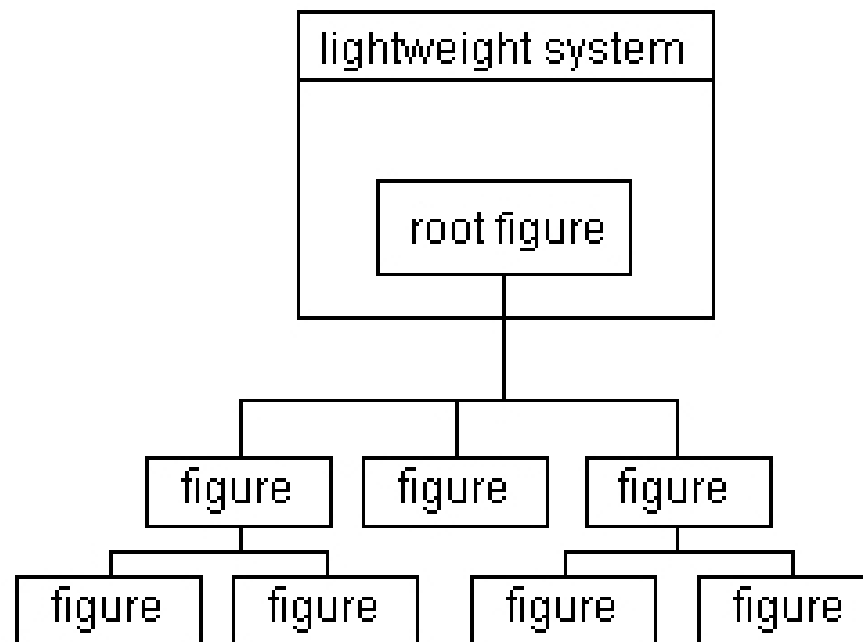


- Interaction Layer
- Model-to-View mapping
- Workbench Integration

- Rendering
- Layout
- Scaling

- Native (SWT) Layer

- **Darstellung der Modellobjekte** in GEF mit Figures.
- **Figures in Baumstruktur.**
- Anzeige der Figures im lightweight system von Draw2D.
- **Figures zeichnen sich selbst** und rekursiv ihre Kinder.



- EditParts wie Figures in **Baumstruktur**.
- **Drei wichtige Methoden** in EditParts:
 - `createFigure()` : **Erstellen der Figure** zu dieser EditPart.
→ Verbindung Controllerschicht ↔ Viewschicht
 - `refreshVisuals()` : **Aktualisieren der Daten** der Viewschicht mit Daten der Modellschicht.
 - `getModelChildren()` : **Liste von Modellklassen**: Logisch Kinder vom zum EditPart korrespondierenden Modellelement.
- **Verbindung Modellschicht ↔ Controller** über EditPartFactory:
 - Neues Modellobjekt erzeugen.
 - In Factory dazu korrespondierenden EditPart suchen.
 - Verbindung knüpfen.

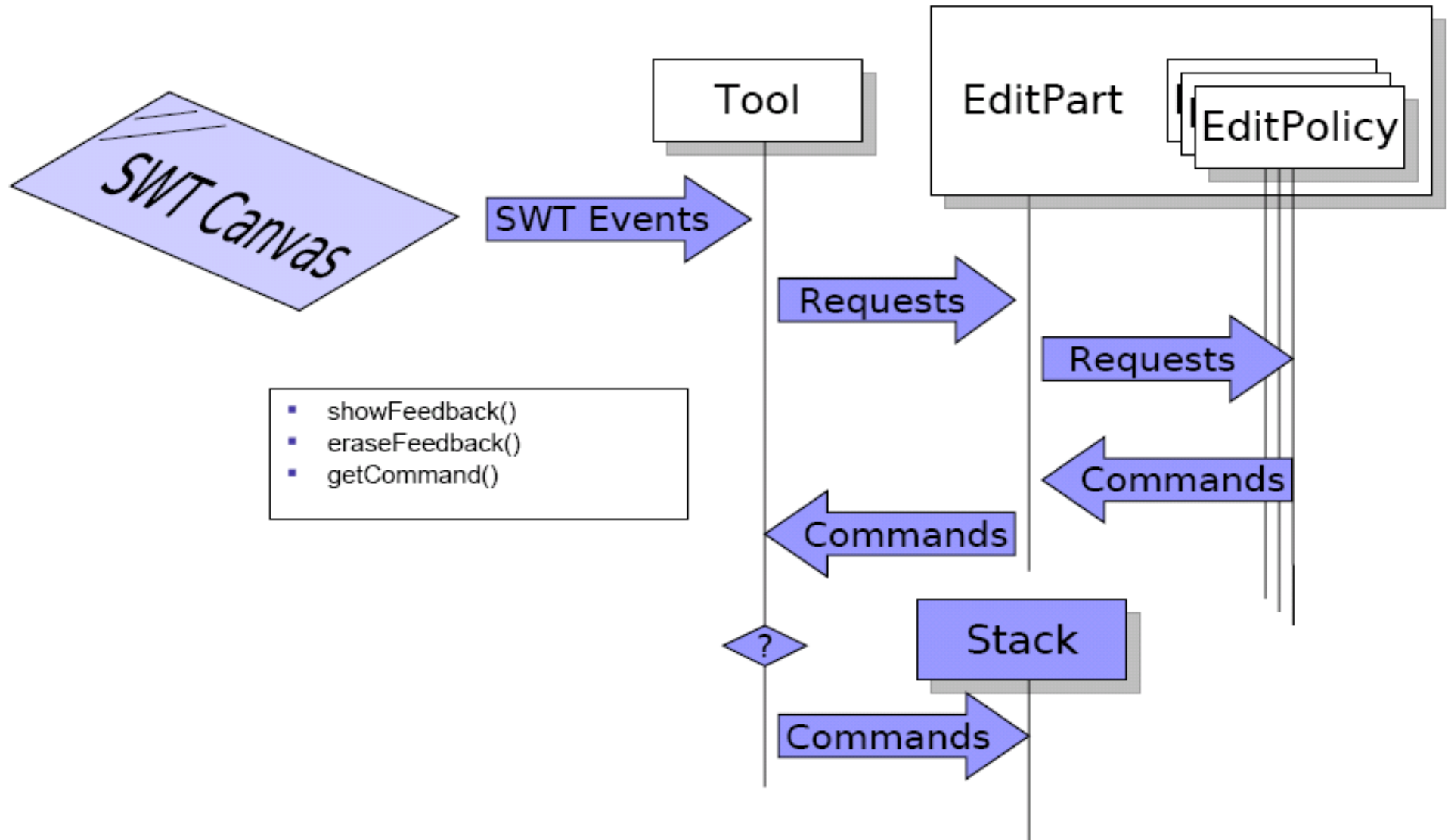
- **Ausgangspunkt:** Änderung findet im Modell statt.
- In EMF sendet Objekt bei Änderung **Notification** an alle registrierten Adapter.
- **Adapter:** EditParts.
- **EditParts:** Bei ihren Modellklassen registrieren.
- Dafür zwei **Methoden:**
 - `activate()` : Nach Erzeugung von EditPart, Registrierung beim entsprechenden Modellelement.
 - `deactivate()` : Wenn EditPart aus EditPartBaum ausgehängt wird, entfernt es Adapter aus dem Modellelement.
- **Notifications an Methode** `notifyChanged(Notification notification)` **senden.**

1.3 Eclipse Modeling Framework (EMF)

Agenda

- Motivation und Überblick
- Eclipse Modeling Framework (EMF)
 - EMF-Modellimport
 - EMOF und Ecore
 - EMF Edit & Codegen
- Graphical Editing Framework (GEF)
 - Model-View-Controller (MVC)-Pattern
 - MVC in GEF
 - Weitere Konstrukte: EditPolicies und Commands
- Nutzung von EMF in GEF
 - Einführung eines Beispiels
 - Beispiel in EMF und GEF
- Zusammenfassung und Ausblick

- **Festlegen der Aufgaben** eines EditParts über EditPolicies.
- **EditPolicies** bekommen Requests.
- **Requests**: Anfragen des Systems, um Aufgabe auszuführen.
- Mit Informationen eines **Requests Command** erzeugen.



- Im Command **Änderungen im Modell** vornehmen.
 - `execute()` : Bei 1. Ausführung des Commands.
 - `undo()` : Rückgängig machen der Aktionen von `execute`.
 - `redo()` : Wiederholen der Aktionen nach `undo`.
- Commands **intern im CommandStack** halten.

- **EditPolicies** in Methode createEditPolicies() von EditPart erzeugen.
- **EditPart**: Verantwortlich View aktuell zu halten.
- EditPolicies behandelt **durch Editieren entstandene Aufgaben**:
 - **Verhindern Einschränkung** durch Einfachvererbung.
 - Übernehmen Aufgaben, die nicht zu EditParts gehören.
 - Erlauben **Bearbeitung dynamisch** zu halten.
 - Werden mithilfe von Roles verwaltet.
 - Behandeln Feedback, Commands, Targeting, etc.
 - **Tipp**: UnexecutableCommand vs. null
- Verwendetes Pattern: **“Pool of Responsibility”**.

- Was wird benutzt um zu **spezifizieren**, welche **commands** auf welche **grafische Elemente** ausführbar sind?
- **Antwort:**
 - **EditParts** benutzen eine Kollektion von **EditPolicy** Instanzen.